

OPERATING SYSTEMS

A Complete Study Material

In-depth explanations with a diagram for every concept

For GATE & Placement Preparation

Based on the GeeksforGeeks OS Tutorial structure

Chapter 1 Basics of Operating Systems

Chapter 2 Process Scheduling

Chapter 3 Process Synchronization

Chapter 4 Deadlock

Chapter 5 Multithreading

Chapter 6 Memory Management

Chapter 7 Disk Management

What's inside each chapter

- Concept intuition (why it exists) • how it works step by step
- a diagram for every idea
 - worked examples
 - trade-offs and key insights
 - comparison tables

Table of Contents

Chapter 1: Basics of Operating Systems

- 1.1 What is an Operating System?
- 1.2 Types of Operating Systems
- 1.3 The Kernel — Heart of the OS
- 1.4 System Calls — The Kernel's API
- 1.5 What Happens When You Turn On a Computer

Chapter 2: Process Scheduling

- 2.1 What Exactly is a Process?
- 2.2 Process States and Transitions
- 2.3 Types of Schedulers
- 2.4 CPU Scheduling Algorithms (FCFS, SJF, RR, Priority, MLQ, MLFQ)
- 2.5 Starvation and Aging

Chapter 3: Process Synchronization

- 3.1 Why We Need Synchronization
- 3.2 The Critical Section Problem
- 3.3 Software Solutions (Peterson's, Dekker's, Bakery)
- 3.4 Semaphores
- 3.5 Monitors and Priority Inversion
- 3.6 Classical IPC Problems

Chapter 4: Deadlock

- 4.1 What is Deadlock?
- 4.2 The Four Necessary Conditions
- 4.3 The Resource Allocation Graph (RAG)
- 4.4 Four Strategies for Handling Deadlock

Chapter 5: Multithreading

- 5.1 Threads — Lightweight Processes
- 5.2 User-Level vs Kernel-Level Threads
- 5.3 Multithreading Models
- 5.4 Benefits of Multithreading — Revisited

Chapter 6: Memory Management

- 6.1 The Memory Management Problem
- 6.2 Logical vs Physical Addresses
- 6.3 Contiguous Memory Allocation

- 6.4 Paging (with TLB)
- 6.5 Segmentation
- 6.6 Virtual Memory and Demand Paging
- 6.7 Page Replacement Algorithms
- 6.8 Thrashing

Chapter 7: Disk Management

- 7.1 File Systems
- 7.2 File Allocation Methods
- 7.3 Directory Structures
- 7.4 Physical Disk Structure
- 7.5 Disk Scheduling Algorithms
- 7.6 Spooling and Buffering
- 7.7 Free Space Management

Appendix: Quick Reference Formulas & Key Points

Chapter 1

Basics of Operating Systems

Before we dive into scheduling, deadlocks, or paging, we need to understand what an OS actually is, why it exists, and how it sits between your program and the hardware. This chapter builds that foundation.

1.1 What is an Operating System?

Imagine you write a simple C program that prints "Hello, World". You didn't tell your program which physical memory address to use for the string, which specific pixels on the screen to light up, or how to talk to the keyboard. Yet somehow it works. The reason is that between your program and the physical hardware sits a piece of software called the **Operating System**.

An OS is **system software that manages hardware resources and provides services to application programs**. It has two audiences: the user (who wants a convenient interface) and the hardware (which needs to be managed efficiently and safely). The OS is essentially a translator, resource allocator, and traffic cop rolled into one.

Analogy: Think of an OS as the manager of a large office building. The tenants (applications) don't need to know which electrical circuit powers their desk, how the elevators work internally, or how mail gets sorted. They just make requests ("turn on lights", "send this email") and the building manager (OS) handles everything: allocating resources fairly, preventing conflicts, and keeping the system running smoothly.

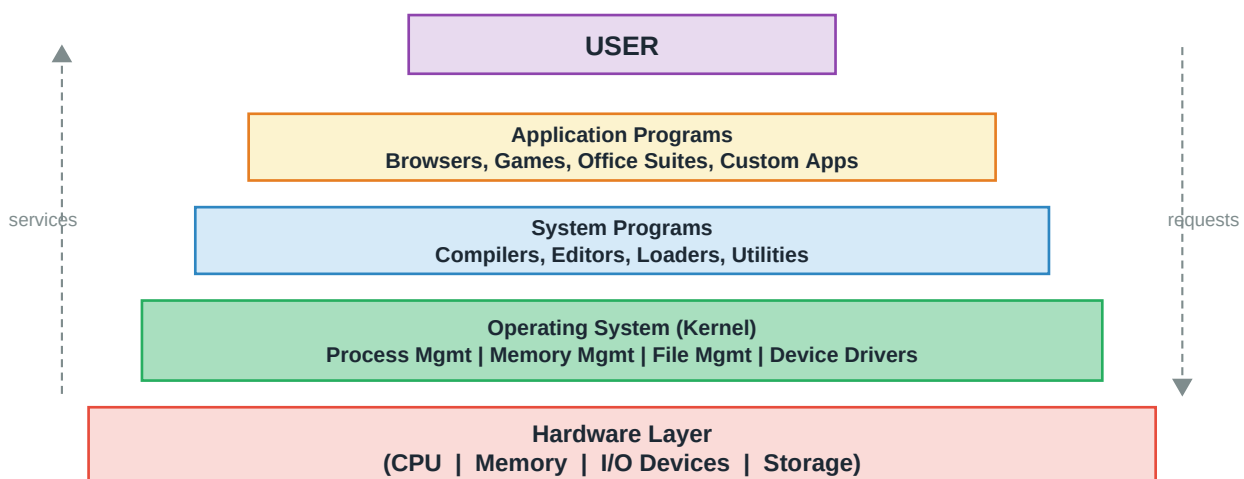


Figure: Layered view — the OS sits between hardware and user software

Why Do We Need an OS?

Without an OS, every program would have to:

- Know the exact hardware layout (impossible to write portable code)

- Manage its own memory, avoiding conflicts with other programs
- Handle every I/O device directly (printer, keyboard, disk...)
- Cooperate voluntarily with other programs (which never works)

The OS solves all of this by providing **abstraction** (a simpler, uniform interface over messy hardware) and **arbitration** (deciding who gets what resource, when).

Goals of an OS

OS design pursues two categories of goals. The **primary goals** are visible to users: convenience (easy to use), program execution (run programs correctly), resource management (share fairly), and security (protect from unauthorized access). The **secondary goals** are internal: efficient use of hardware, reliability (graceful failure handling), and modularity (easy to maintain and extend).

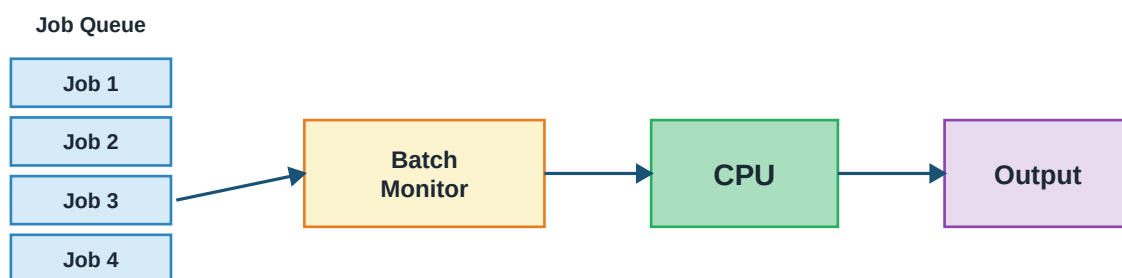
Key Insight: There's a tension between these goals. Convenience often costs efficiency (a friendly GUI uses more CPU than a bare command line). Reliability often costs performance (extra checks slow things down). Every OS design decision is a trade-off.

1.2 Types of Operating Systems

OSes have evolved to solve different problems. Understanding the types tells you the history of computing and why modern OSes look the way they do.

1. Batch Operating System

In the earliest days of computing, computers were expensive and users were cheap. A user would submit a "job" (program + input data) on punched cards, hand it to an operator, and come back hours later for the output. The **Batch OS** grouped similar jobs together and processed them one after another with no user interaction.



One job runs at a time to completion; no user interaction during execution

Figure: Batch OS — jobs sit in a queue and execute one at a time

The problem was CPU utilization. Whenever a job did I/O (like reading the next card), the CPU sat idle — wasting expensive machine time. The response time was also terrible: if your job was 50th in the queue, you waited for 49 other jobs to complete.

Where you still see batch processing: Nightly bank statement generation, insurance claim processing, payroll runs — situations where jobs are similar, don't need interactivity, and can be scheduled for off-peak hours.

2. Multi-Programming Operating System

The insight behind multi-programming is simple but powerful: **when one program is waiting for I/O, another program can use the CPU**. Instead of running one job at a time, keep several jobs in memory and switch to another whenever the current one blocks on I/O. The CPU stays busy, throughput goes up.

Multiprogramming: CPU switches to another job during I/O wait

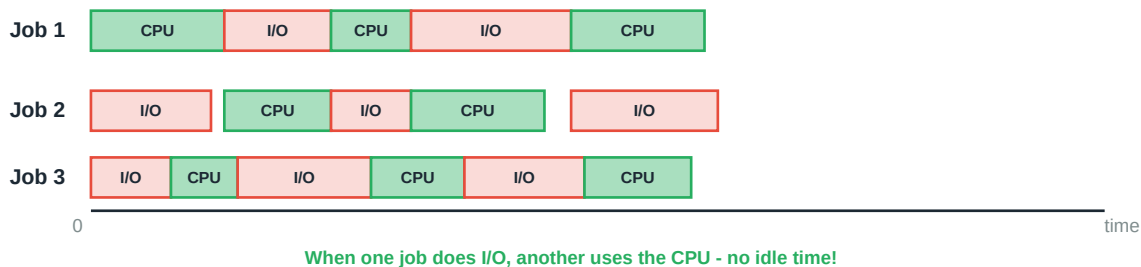


Figure: Multiprogramming — CPU switches to another job during I/O wait

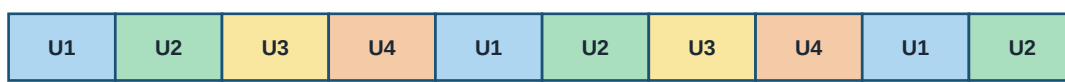
This required new OS capabilities: memory management (to hold multiple programs safely), CPU scheduling (to pick which one to run next), and I/O management (to know when a program's I/O is done). This complexity is why modern OSes are as sophisticated as they are — every one of those features has a chapter or two dedicated to it in this book.

Key Insight: Multiprogramming is about **utilization**, not **responsiveness**. A single job might not run any faster — it might even take longer because of switching overhead. But the total work done per hour increases dramatically.

3. Time-Sharing (Multi-tasking) OS

Multi-programming made the machine efficient but didn't help users — they still had to submit jobs and wait. **Time-sharing** takes multi-programming one step further: it gives each user (or process) a small time slice called a **quantum** (typically 10-100 milliseconds), then switches to the next one. Switching happens so fast that everyone feels like they have the computer to themselves.

Time-Sharing: Each user/process gets a small time slice (quantum)



Quantum = fixed time slice (e.g., 10ms) | Users feel simultaneous execution

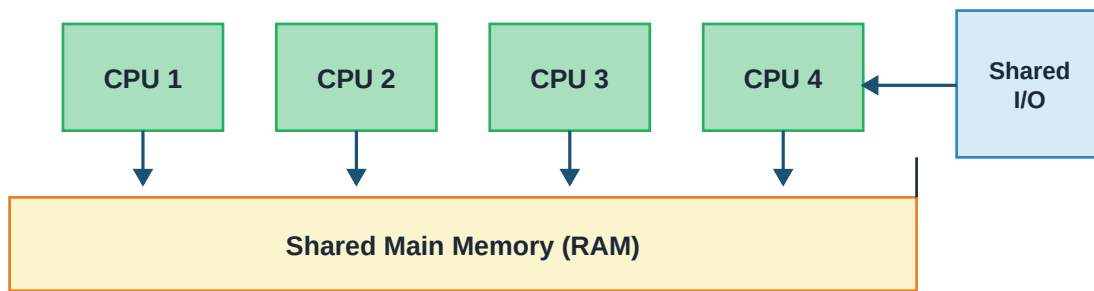
Figure: Time-sharing — small quanta rotate the CPU among users/processes

This is what your laptop does right now. You have a browser, a music player, and a text editor all "running at once" — they aren't really; the CPU is switching between them dozens of times per second. Time-sharing OSes gave birth to interactive computing.

4. Multi-Processing OS

So far we've assumed one CPU. What if the machine has multiple CPUs (or one CPU with multiple cores)? A **multi-processing OS** can execute processes truly in parallel — not just interleaved but simultaneously on

different processors.



Multiple CPUs share memory and I/O; increases throughput and provides fault tolerance

Figure: Multi-processing — multiple CPUs share memory and I/O for true parallelism

Two main organizations exist. In **Symmetric Multi-Processing (SMP)**, all processors are equal peers, any of them can run any process, and they share memory. Modern desktops and servers use SMP. In **Asymmetric Multi-Processing (AMP)**, one master processor controls the others which do specialized work — used in some embedded systems.

Key Insight: Multi-processing gives you real parallelism but adds new problems: cache coherence between CPUs, synchronization on shared data, and scheduling that avoids moving processes needlessly between CPUs.

5. Distributed Operating System

Push the idea further: what if the processors aren't in the same box but connected over a network? A **distributed OS** makes multiple independent computers appear as a single unified system to users. Each machine has its own CPU and memory, but they cooperate through message passing.

Distributed OS: Independent computers appear as one unified system

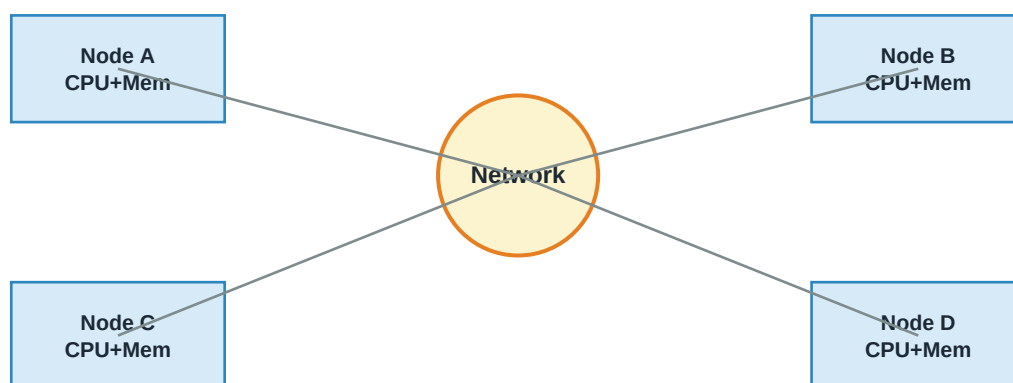


Figure: Distributed system — multiple nodes over a network appear as one system

Distributed systems are the backbone of cloud computing, but they're notoriously hard to build. Network delays are unpredictable, machines can fail independently, and there's no global clock, so knowing "what happened first" becomes surprisingly subtle. This is a whole field of study on its own.

6. Real-Time OS (RTOS)

In some systems, being correct isn't enough — you must also be **on time**. If your car's airbag controller detects a crash but takes 500ms to fire the airbag, the airbag was correct but useless. A **Real-Time OS**

guarantees response within strict time bounds.

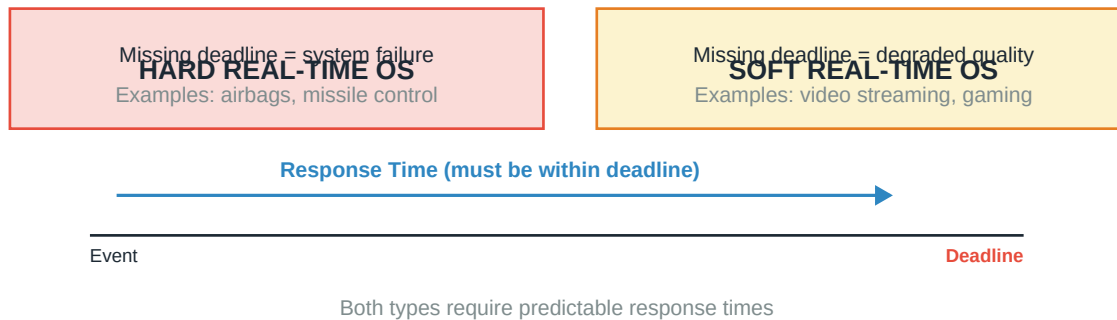


Figure: Hard vs Soft RTOS — how strictly the deadline must be met

Hard RTOS is used where missing a deadline is catastrophic: missile guidance, medical devices, industrial control. **Soft RTOS** is used where missing a deadline degrades quality but doesn't break the system: video playback, gaming, VoIP. In soft real-time, occasional glitches are annoying but acceptable; in hard real-time, they can kill people.

RTOSes avoid features that add unpredictability. They often skip virtual memory (paging can cause unpredictable delays) and use scheduling algorithms with guaranteed response times (like Rate Monotonic).

7. Network Operating System

A **Network OS** runs on a server and provides shared services — files, printers, authentication — to other computers on a local network. Unlike a distributed OS, users are aware of the network topology and explicitly connect to specific servers. Examples: Windows Server, various Linux server distributions.

8. Mobile Operating System

Mobile OSes (Android, iOS) evolved from desktop OSes but with constraints: limited battery, limited memory, touch input instead of keyboard/mouse, and always-on connectivity. They emphasize energy efficiency, app sandboxing for security, and background task limits to preserve battery life.

1.3 The Kernel — Heart of the OS

When we say "OS", we sometimes mean the entire package (including utilities, GUI, and drivers), and sometimes just the innermost part that talks to hardware. That innermost part is the **Kernel**. It's the first program loaded when you boot, and the last to shut down. Everything else runs on top of it.

The kernel runs in a special **privileged mode** (also called kernel mode or supervisor mode) that lets it execute any CPU instruction and access any memory location. User programs run in **user mode** with restricted privileges — they can't directly access hardware or step on each other's memory. This dual-mode operation is what makes modern OSes safe.

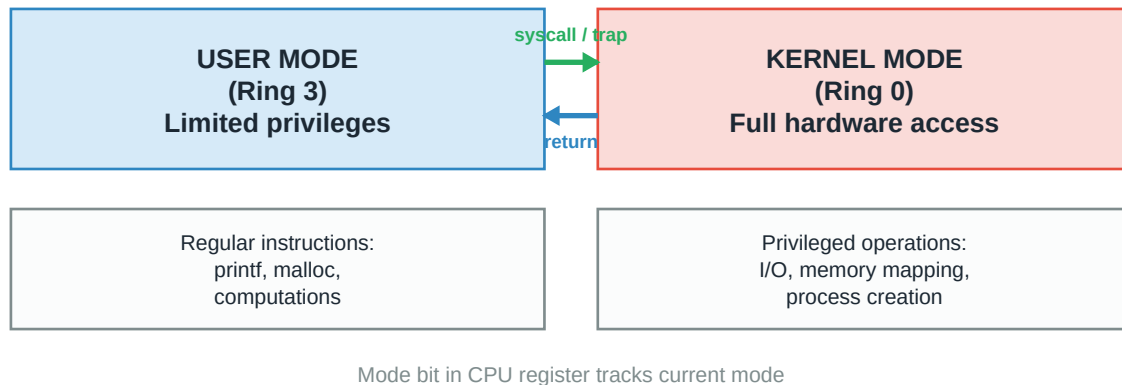


Figure: User mode vs kernel mode — enforced by hardware

Types of Kernel Architecture

How much should the kernel do? This turns out to be a controversial design decision, with different OSes making different choices.

Kernel Architectures

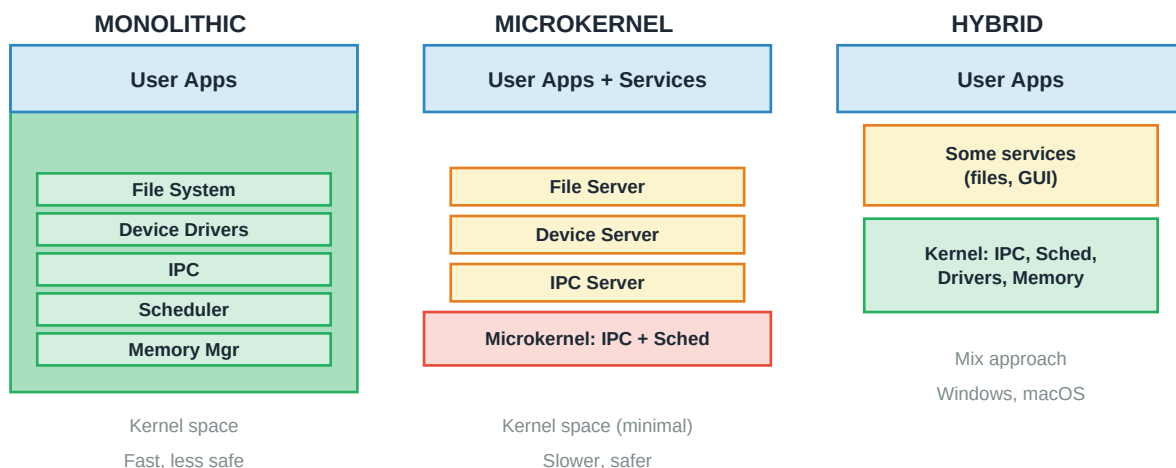


Figure: Three kernel architectures — monolithic, micro, and hybrid

Monolithic Kernel

Everything — file systems, drivers, network stack, scheduler, memory manager — runs together in kernel mode as one big program. Communication between components is direct function calls, so it's **very fast**. The downside: a bug in any driver can crash the entire OS, because everything shares the same address space. Linux is monolithic (though it uses loadable modules to feel modular).

Microkernel

Only the absolute essentials — IPC, basic scheduling, and low-level memory management — run in kernel mode. File systems, drivers, and network stack run as **user-space servers** that communicate via messages. Advantages: much safer (a crashed driver doesn't take down the OS) and easier to develop. Disadvantage: slower, because every service call becomes a message-passing operation. QNX and Minix are microkernels.

Hybrid Kernel

A compromise: performance-critical services (like graphics) run in kernel mode for speed, while others run as user-space services for safety. Windows NT and macOS use hybrid kernels.

Key Insight: The monolithic-vs-microkernel debate famously happened between Linus Torvalds (monolithic) and Andrew Tanenbaum (microkernel) in 1992. Decades later, both approaches are still alive — the answer is 'it depends on your goals'.

Functions of the Kernel

- **Process management:** Create/destroy processes, schedule the CPU, handle context switches.
- **Memory management:** Allocate/free memory, implement virtual memory, protect processes from each other.
- **File system management:** Organize files on disk, translate between file names and disk locations.
- **Device management:** Provide a uniform interface to diverse hardware through device drivers.
- **Security & access control:** Authenticate users, check permissions on every operation.
- **Inter-process communication (IPC):** Provide pipes, sockets, shared memory, and message queues.

1.4 System Calls — The Kernel's API

If user programs can't directly access hardware, how do they do anything useful? The answer: they ask the kernel to do it for them, through **system calls**.

A system call is like calling a function, except the function lives inside the kernel and requires a special CPU instruction to invoke. That special instruction (called a **trap** or software interrupt) does two things: it switches the CPU into kernel mode, and it jumps to a predetermined entry point in the kernel.

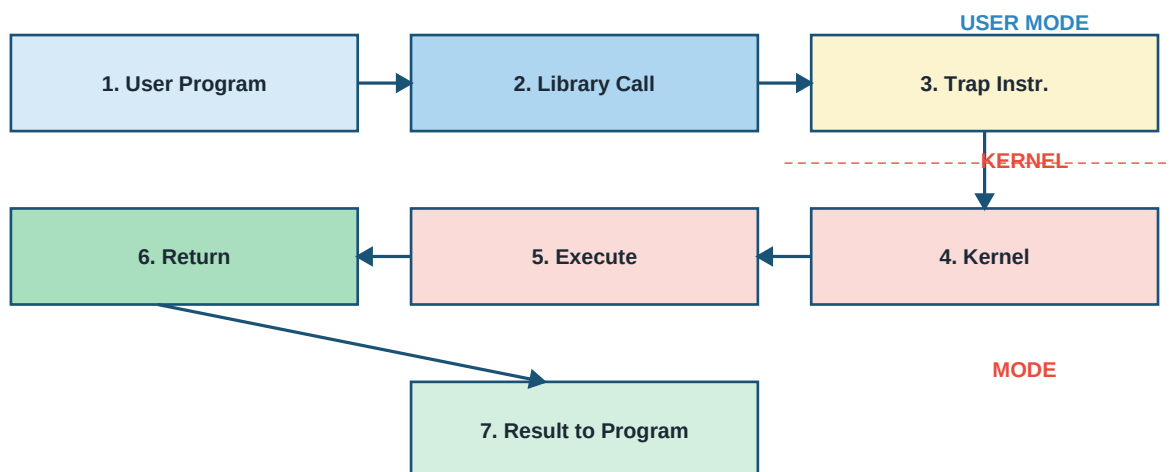


Figure: System call flow — user code triggers a trap, kernel handles it, returns result

How System Calls Work — Step by Step

When you write `printf("hello")` in C, here's what actually happens:

- **1.** Your call to `printf` is a normal function call to the C library.
- **2.** The C library formats your string and eventually calls `write()`, which is a system call wrapper.

- **3.** The wrapper places the system call number in a specific CPU register and executes a trap instruction (like `syscall` on x86-64).
- **4.** The CPU switches to kernel mode and jumps to the kernel's system call handler.
- **5.** The kernel looks up the `syscall` number, dispatches to the right function (write in this case), and does the actual work.
- **6.** The kernel switches back to user mode and returns the result.
- **7.** Your program continues, unaware of the kernel-mode gymnastics.

Important: A system call is NOT the same as a context switch. A `syscall` is a **mode switch** — same process, different privilege. A context switch is when the CPU stops running one process and starts running another (which requires saving/restoring the entire PCB). Every context switch requires a mode switch, but a mode switch doesn't always trigger a context switch.

Categories of System Calls

Category	Purpose	Example System Calls
Process Control	Create, terminate, wait for processes	<code>fork()</code> , <code>exec()</code> , <code>exit()</code> , <code>wait()</code>
File Management	Open, read, write, close files	<code>open()</code> , <code>read()</code> , <code>write()</code> , <code>close()</code> , <code>lseek()</code>
Device Management	Request/release devices, I/O operations	<code>ioctl()</code> , <code>read()</code> , <code>write()</code>
Information Maintenance	Get/set system state and time	<code>getpid()</code> , <code>time()</code> , <code>alarm()</code> , <code>sleep()</code>
Communication (IPC)	Create pipes, sockets, shared memory	<code>pipe()</code> , <code>socket()</code> , <code>shmget()</code> , <code>mmap()</code>
Protection	Set permissions, access control	<code>chmod()</code> , <code>chown()</code> , <code>umask()</code>

1.5 What Happens When You Turn On a Computer

You press the power button. In under a minute, you're at a login screen. What actually happened in that time? A carefully orchestrated sequence called the **boot process**.



Boot sequence: hardware initialization to user-ready system

Figure: Boot process — from power-on to a usable system

- **1. Power on:** Electricity reaches the motherboard. The CPU starts executing instructions from a fixed address in ROM (read-only memory).
- **2. BIOS/UEFI:** That fixed address holds firmware (BIOS on older PCs, UEFI on modern ones). It runs POST (Power-On Self Test) to check that CPU, RAM, and basic devices are working.
- **3. Bootloader:** BIOS/UEFI finds a bootable device (usually the primary disk) and loads a small program called the **bootloader** (e.g., GRUB on Linux) from a fixed location.
- **4. Kernel loading:** The bootloader loads the OS kernel into memory and transfers control to it.
- **5. Kernel initialization:** The kernel initializes memory management, sets up interrupt handlers, mounts the root filesystem, and starts device drivers.

- **6. First user process:** The kernel launches the first user-space process (called `init` or `systemd` on Linux). Every other process is a descendant of this one.
- **7. System services and login:** Init starts background services (network, logging, etc.) and finally shows the login screen.

Key Insight: The reason the boot process is layered like this is **portability and evolution**. The tiny piece of code in ROM only needs to know how to load a bootloader. The bootloader only needs to know how to load a kernel. Each layer can be updated independently, and different OSes can coexist on the same machine.

Chapter 2

Process Scheduling

The CPU is the most valuable resource in the system. Deciding which process gets it, and for how long, is one of the OS's most important jobs. This chapter covers processes, their states, and the classic CPU scheduling algorithms.

2.1 What Exactly is a Process?

You'll hear "process" and "program" used interchangeably in casual conversation, but they're not the same thing. A **program** is a passive entity — the executable file on disk. A **process** is an active entity — a program in execution, with its own memory, CPU state, and open files.

Analogy: A program is a recipe in a cookbook. A process is you actually cooking that dish — with ingredients measured out, pans on the stove, and specific progress through the steps. You can cook the same recipe twice at once (two processes running the same program), and each will have its own state.

Memory Layout of a Process

Every process gets its own private memory divided into segments. Understanding this layout tells you where variables live, how stack overflows happen, and why one process can't peek into another's data.

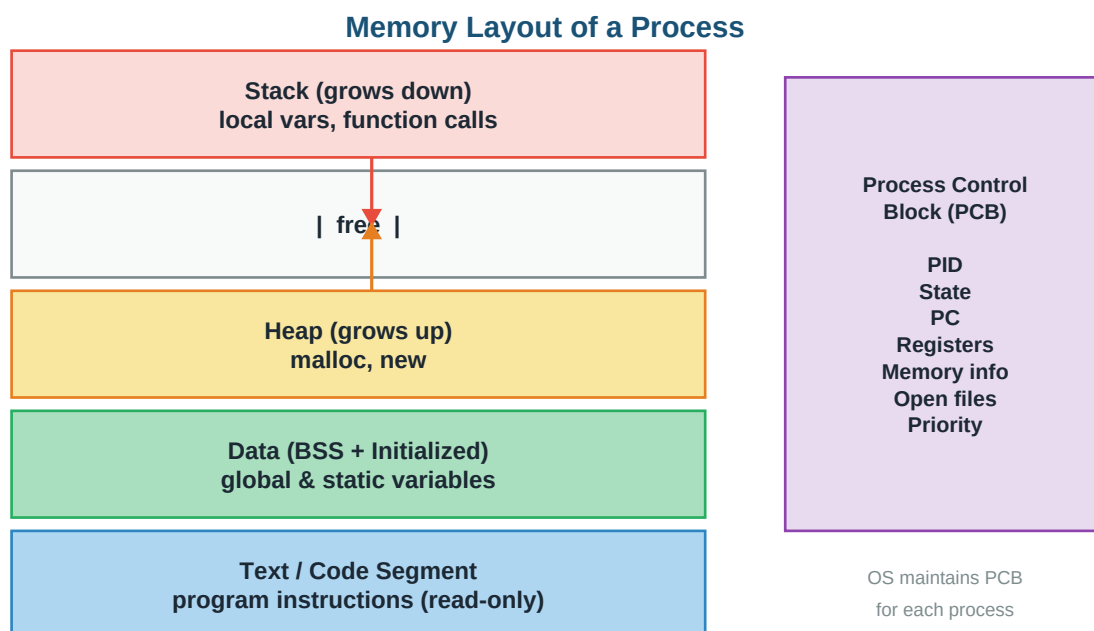


Figure: Memory layout — text (code), data, heap, stack, plus PCB in kernel space

- **Text segment:** The compiled machine code. Read-only, so accidental writes are caught by the hardware.
- **Data segment:** Global and static variables. Split into initialized (values set at compile time) and BSS (zeroed at startup).
- **Heap:** Dynamically allocated memory (malloc, new). Grows upward as you allocate more.

- **Stack:** Function call frames — local variables, return addresses, saved registers. Grows downward. When it collides with the heap, you get a stack overflow.

Process Control Block (PCB)

The OS needs to keep track of every process. For each one, it maintains a data structure called the **Process Control Block**. The PCB is the process's identity from the OS's perspective — everything the OS knows about that process lives here.

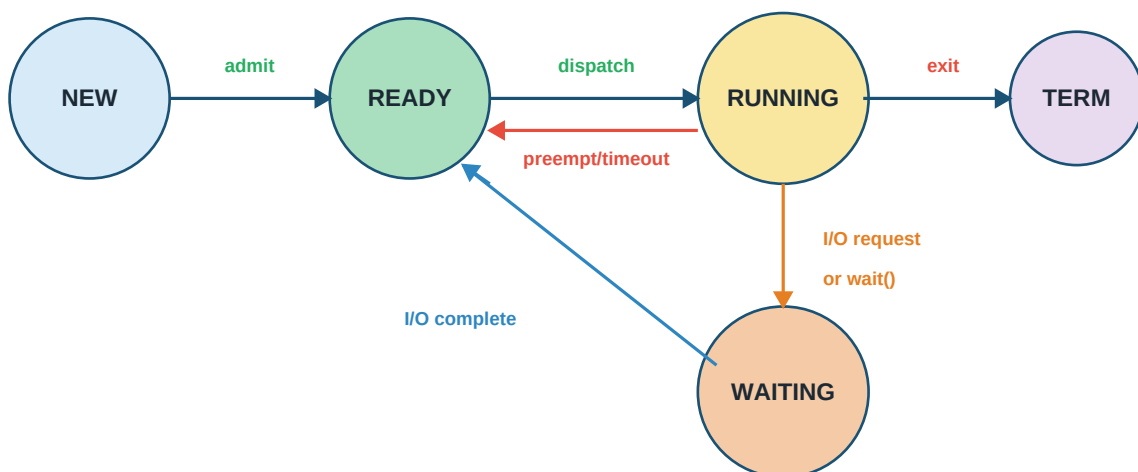
The PCB contains:

- **Process ID (PID):** A unique integer identifying this process.
- **Process state:** New, Ready, Running, Waiting, Terminated (covered next).
- **Program counter:** Address of the next instruction to execute when this process resumes.
- **CPU registers:** Contents of all CPU registers — saved so we can restore the process later.
- **Memory management info:** Page tables, segment tables, base and limit registers.
- **Scheduling info:** Priority, pointers into scheduling queues, time slice remaining.
- **I/O status:** List of open files, allocated I/O devices, pending I/O requests.
- **Accounting info:** CPU time used, wall-clock time since start, page fault count.

Key Insight: When the OS switches from process A to process B, it saves A's CPU registers into A's PCB and loads B's registers from B's PCB. This is what a context switch physically is.

2.2 Process States and Transitions

A process doesn't run continuously from start to finish. It moves through several **states** during its lifetime, reflecting whether it's currently running, ready to run, or waiting for something.



Process moves between states based on CPU scheduling and events
PCB is updated on every state transition (context switch)

Figure: Process state diagram — every transition has a specific cause

The Five States

- **New:** The process is being created. The OS is setting up the PCB and allocating initial memory.

- **Ready:** The process is ready to run and waiting for the CPU. It's sitting in the ready queue.
- **Running:** The process is currently executing on a CPU.
- **Waiting (Blocked):** The process is waiting for some event — usually I/O completion, but also things like waiting on a semaphore or child process.
- **Terminated:** The process has finished (or was killed). Its PCB is being cleaned up.

The Transitions

Every state change has a specific cause. Understanding the transitions is more important than memorizing the states.

- **New → Ready:** OS finishes creating the process; it's now eligible to run.
- **Ready → Running:** The scheduler picks this process and dispatches it to the CPU.
- **Running → Ready:** Time slice expired (preempted) or a higher-priority process arrived. The process is still able to run but had to give up the CPU.
- **Running → Waiting:** The process made a blocking system call (e.g., `read()` waiting for disk).
- **Waiting → Ready:** The event the process was waiting for happened (I/O finished, semaphore released).
- **Running → Terminated:** The process finished normally or was killed.

Important: Note: there is no direct Waiting → Running transition. A waiting process must first become Ready, then be chosen by the scheduler. The scheduler is the only path to Running.

2.3 Types of Schedulers

The OS has multiple schedulers operating at different time scales. Each has a distinct job.

Three Types of Schedulers

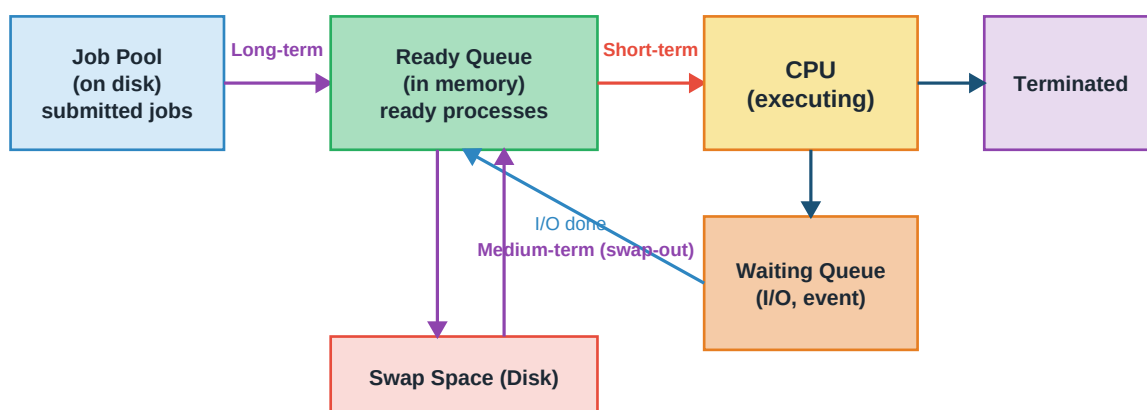


Figure: Three schedulers — long-term, short-term, and medium-term

Long-Term Scheduler (Job Scheduler)

Decides which jobs are admitted into the system for processing. It controls the **degree of multiprogramming** — how many processes are in memory at once. Runs infrequently (every few seconds or when a process finishes). Its goal is to maintain a healthy mix of CPU-bound and I/O-bound processes so both the CPU and I/O devices stay busy.

Interactive systems (like your laptop) often skip the long-term scheduler entirely — as soon as you double-click an icon, the process is created and enters the ready queue.

Short-Term Scheduler (CPU Scheduler)

This is the star of this chapter. Every time the CPU becomes free (a process finishes, blocks, or is preempted), the short-term scheduler picks the next process from the ready queue. It runs **very frequently** — often every few milliseconds — so it must be extremely fast. All the algorithms in Section 2.4 are short-term scheduling algorithms.

Medium-Term Scheduler

Handles **swapping** — temporarily moving processes out of main memory to disk when memory is tight. When memory frees up, the swapped-out process can be brought back. This is a way to reduce the degree of multiprogramming without killing processes.

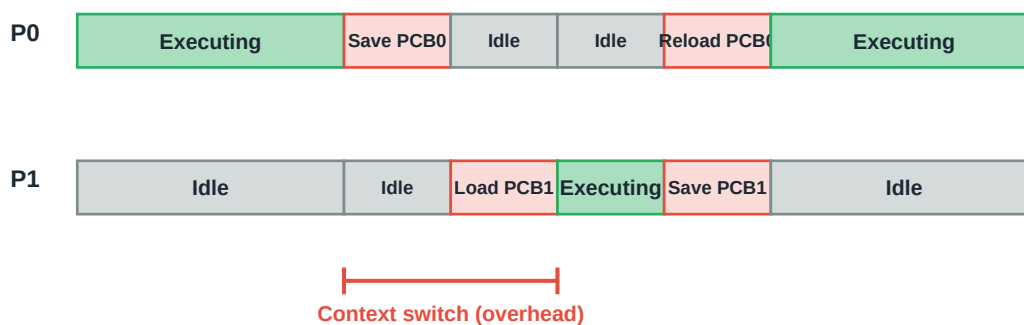
Scheduler vs Dispatcher

These two terms are often confused. The **scheduler** is the algorithm that decides WHO runs next. The **dispatcher** is the piece of code that actually gives control to the chosen process. Dispatching involves: switching context (saving old PCB, loading new PCB), switching to user mode, and jumping to the resumption address. The time it takes is called **dispatch latency**.

Context Switching — the Cost of Multitasking

Every time the OS switches from one process to another, it must save the current process's state and load the next process's state. This is **pure overhead** — no useful work is done.

Context Switch: Overhead of Multitasking



Both P0 and P1 do NO useful work during context switch - it's pure overhead

Figure: Context switch — both processes are idle during the switch itself

A context switch typically takes 1-1000 microseconds depending on hardware. This is why designing scheduling algorithms is a delicate balance: switch too often and you drown in overhead; switch too rarely and responsiveness suffers.

2.4 CPU Scheduling Algorithms

Now the heart of the chapter. Given a queue of ready processes, which one runs next? Different algorithms optimize different things. Before we look at them, let's define what we're optimizing.

Key Metrics

- **CPU Utilization:** Fraction of time the CPU is busy. Want to maximize.
- **Throughput:** Number of processes completed per unit time. Want to maximize.
- **Turnaround Time:** Total time from process arrival to completion. Want to minimize.
- **Waiting Time:** Time spent waiting in the ready queue (not running, not doing I/O). Want to minimize.
- **Response Time:** Time from arrival until the process first starts producing output. Want to minimize for interactive systems.

Turnaround Time (TAT) = Completion Time - Arrival Time

Waiting Time (WT) = Turnaround Time - Burst Time

Response Time (RT) = First Run Time - Arrival Time

Preemptive vs Non-Preemptive

In **non-preemptive** scheduling, once a process gets the CPU, it keeps it until it voluntarily gives it up (by finishing or doing I/O). In **preemptive** scheduling, the OS can forcibly take the CPU away — usually when the time slice expires or a higher-priority process becomes ready.

Preemptive scheduling gives better responsiveness (short processes don't get stuck behind long ones) but adds overhead (more context switches) and creates synchronization issues (a process can be interrupted in the middle of updating shared data).

2.4.1 First Come First Serve (FCFS)

The simplest possible algorithm: run processes in the order they arrive. It's **non-preemptive** — once a process starts, it runs to completion.

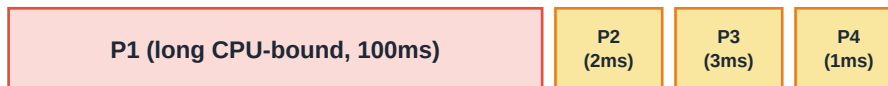
FCFS Example: P1(24), P2(3), P3(3) — all arrive at t=0



Figure: FCFS Gantt chart — waiting times: P1=0, P2=24, P3=27 → avg = 17

The problem is visible immediately: P2 and P3 (each only 3 units long) have to wait 24+ units behind P1. If P1, P2, P3 had arrived in a different order, the average waiting time would drop dramatically.

The Convoy Effect (Problem with FCFS)



Waiting queue direction ->

Small processes wait a long time behind a big one, hurting throughput

Analogy: One slow truck holding up a line of fast cars

Figure: Convoy effect — one big process at the front holds up everyone behind

This is called the **Convoy Effect**: short I/O-bound processes get stuck behind long CPU-bound ones, hurting overall throughput. FCFS is simple and starvation-free, but its performance is often poor. It's a good baseline to compare other algorithms against.

2.4.2 Shortest Job First (SJF)

If long processes cause problems, run **short ones first**. SJF picks the process with the smallest CPU burst time. It has a beautiful property: it's **provably optimal** in terms of average waiting time among non-preemptive algorithms.

SJF: same processes, ordered by burst time



Figure: SJF Gantt chart — average waiting time drops significantly

SJF has two flavors. **Non-preemptive SJF**: once a process starts, it runs to completion — even if a shorter one arrives. **Preemptive SJF**, also called **Shortest Remaining Time First (SRTF)**: if a new process arrives with a shorter burst than what remains of the current process, preempt immediately.

Important: SJF has two big problems. First, **starvation**: if short jobs keep arriving, long jobs may never run. Second, **you don't know the burst time in advance** — it hasn't happened yet. Real systems estimate the next burst using exponential averaging of past bursts, but it's still just a guess.

2.4.3 Round Robin (RR)

Give each process a fixed **time quantum** (usually 10-100ms). When the quantum expires, preempt the process and put it at the back of the ready queue. Then run the next process for one quantum, and so on.

Round Robin: quantum=4, P1(20), P2(6), P3(4)

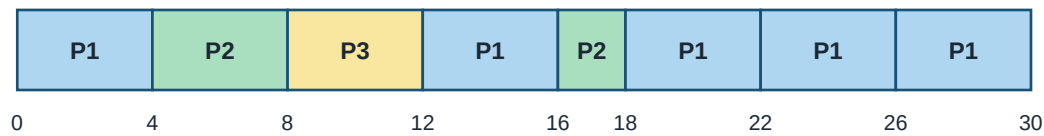


Figure: Round Robin Gantt chart — CPU rotates fairly among processes

The quantum size is critical. Too large and RR degenerates into FCFS (a process finishes before its quantum expires). Too small and you waste time on context switches. Rule of thumb: 80% of CPU bursts should be shorter than the quantum.

Key Insight: Round Robin is the default for time-sharing systems because it guarantees no starvation and gives good response time. Its trade-off is worse throughput than SJF and no priority handling.

2.4.4 Priority Scheduling

Assign each process a priority number. The CPU goes to the highest-priority ready process. Priorities can be set explicitly by the user, or computed by the OS from factors like process type, resource usage, and I/O behavior.

Note: in most systems, a **lower number means higher priority** — this is a convention that catches many students off guard. Priority scheduling can be preemptive or non-preemptive; SJF is actually a special case of priority scheduling where $\text{priority} = 1/\text{burst_time}$.

Important: Priority scheduling can cause **starvation**. If high-priority processes keep arriving, low-priority processes may wait forever. The fix is **aging**: gradually increase the priority of processes that have been waiting a long time. Eventually every process becomes high-priority enough to run.

2.4.5 Multilevel Queue Scheduling

Sometimes you want different policies for different kinds of processes — interactive processes need responsiveness, batch jobs need throughput, real-time processes need guaranteed CPU. Multilevel Queue splits the ready queue into several separate queues (e.g., interactive, batch, system), each with its own scheduling algorithm. A higher-level scheduler decides which queue gets the CPU next (typically fixed priority between queues).

2.4.6 Multilevel Feedback Queue (MLFQ)

MLFQ extends multilevel queue with something clever: **processes can move between queues** based on their behavior. A process starts in a high-priority queue with a short quantum. If it uses the full quantum (behaves like a CPU-bound process), it drops to a lower queue with a longer quantum. If it blocks quickly (behaves like I/O-bound), it stays high.

This automatically favors interactive processes (which typically do short CPU bursts before blocking on I/O) while still processing CPU-bound work — the CPU-bound processes get long quanta so they aren't context-switched constantly. MLFQ is the most general scheduling framework and is used by many modern OSes.

Comparison of Scheduling Algorithms

Algorithm	Type	Starvation?	Optimal for	Downside
FCFS	Non-preemptive	No	Simplicity	Convoy effect
SJF (non-preempt)	Non-preemptive	Yes	Avg waiting time	Needs burst estimate
SRTF	Preemptive	Yes	Avg waiting time	Frequent context switch
Round Robin	Preemptive	No	Response time	Quantum tuning matters
Priority	Either	Yes (fix with aging)	Meeting priorities	Starvation, priority inversion
MLFQ	Preemptive	No (with aging)	General purpose	Complex to tune

2.5 Starvation and Aging

Starvation (also called indefinite blocking) happens when a process waits indefinitely because other processes are constantly preferred over it. It's not a deadlock — the starving process could run if given a chance, but it never gets one.

Aging is the standard cure: as a process waits, its priority gradually increases. Eventually, even the lowest-initial-priority process becomes the highest-priority process in the ready queue, guaranteeing it runs. This adds a bit of complexity but makes the system fair.

Example: In a priority system where priorities range 0-127 (0 = highest), we might add 1 to a process's priority number for every 10 seconds it waits — no wait, we'd SUBTRACT 1 to raise priority. After 1280 seconds, even a process that started at priority 127 would reach priority 0. In practice systems are less patient — aging happens over seconds, not minutes.

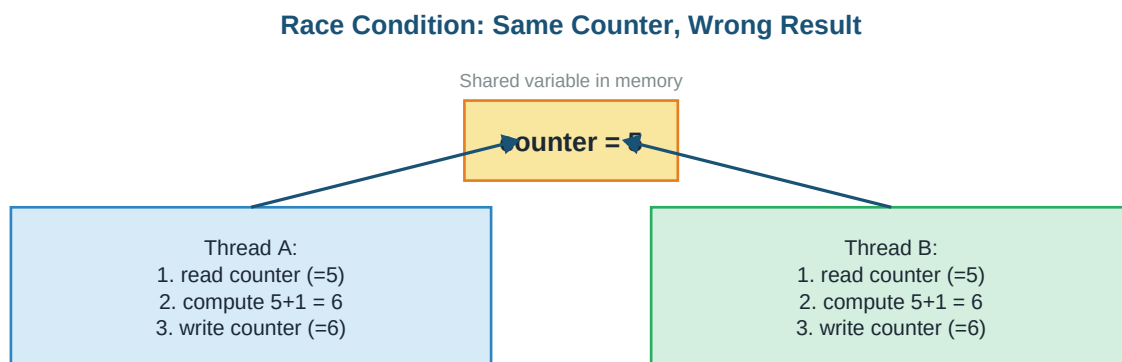
Chapter 3

Process Synchronization

When multiple processes or threads share data, chaos can break loose in surprising ways. This chapter is about coordination — ensuring that concurrent programs produce correct results, no matter how the scheduler interleaves them.

3.1 Why We Need Synchronization

Concurrency (multiple threads of execution running at overlapping times) is powerful but treacherous. When two processes access shared data without coordination, you can get results that would be impossible in a sequential program. Let's see this in action.



Expected: $5 + 1 + 1 = 7$ | Actual (with race): 6

Both threads read old value before either writes -> one update is lost

Figure: Race condition — two threads increment a counter, one update is lost

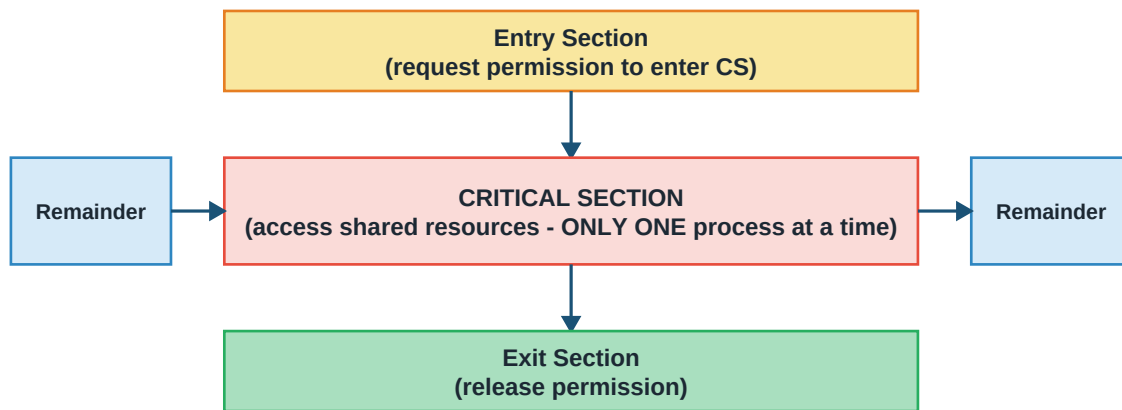
In the example above, both threads read the counter's value (5) at the same time, both compute 6, and both write 6. Two increments happened, but the counter only went up by 1. This is a **race condition**: the outcome depends on the exact order of operations, and the order isn't guaranteed.

Analogy: Two people share a bank account with \$100. Both check the balance (see \$100), both decide to withdraw \$80. Both succeed because at the moment of check, \$100 was available. Now the account is at -\$60 instead of the -\$60 that would happen sequentially — actually it's worse: sequentially the second withdrawal would fail. Concurrent access without coordination lets impossible things happen.

3.2 The Critical Section Problem

The part of a program that accesses shared data is called the **critical section**. Our goal is to ensure that at most one process is in its critical section at any time — this is **mutual exclusion**.

Structure of a Program with Critical Section



Solution must guarantee: Mutual Exclusion, Progress, Bounded Waiting

Figure: Critical section structure — entry, CS, exit, remainder

Three Requirements for a Correct Solution

Any valid solution to the critical section problem must satisfy three properties:

- **1. Mutual Exclusion:** At most one process can be in its critical section at a time. This is the core requirement — without it, we have the race condition we're trying to prevent.
- **2. Progress:** If no process is in the critical section, and some processes want to enter, then the selection of who enters next must not be postponed indefinitely. In other words, the algorithm can't block progress by processes that aren't even trying to enter.
- **3. Bounded Waiting:** There must be a bound on how many times other processes can enter their critical sections after a process has requested entry and before that request is granted. This prevents starvation.

3.3 Software Solutions

Peterson's Algorithm (Two Processes)

Peterson's Algorithm is a classic software-only solution for mutual exclusion between two processes. It uses two shared variables:

- `flag[2]` — a boolean array where `flag[i]=true` means process *i* wants to enter its critical section.
- `turn` — an integer indicating whose turn it is to enter.

Process P_i runs the following code:

```

// Entry section
flag[i] = true; // I want to enter
turn = j; // But I let the other go first
while (flag[j] && turn == j) // Wait if other also wants and it's their turn
; // spin

// CRITICAL SECTION

// Exit section
  
```

```
flag[i] = false; // I'm done
```

The trick is $\text{turn} = j$: process P_i politely says "the other one can go first". If both processes reach this point simultaneously, one of them writes turn last, and that value determines the winner. P_i only proceeds when EITHER P_j isn't interested ($\text{flag}[j] == \text{false}$) OR it's P_i 's turn.

Important: Peterson's Algorithm assumes atomic reads and writes of individual variables and sequential memory consistency. On modern multi-core CPUs with out-of-order execution and caching, it needs explicit memory barriers to work correctly. It's a great teaching tool but almost never used in production.

Dekker's Algorithm

Dekker's Algorithm was the first known correct software solution to mutual exclusion (Dijkstra credited Dekker in 1965). It's more complex than Peterson's and predates it by about 15 years, but Peterson's is cleaner and equally correct for two processes.

Bakery Algorithm (N Processes)

Peterson's works only for two processes. Lamport's **Bakery Algorithm** generalizes to any number. The idea comes from a bakery: when you enter, you take a numbered ticket, and customers are served in ticket order. Each process picks a number larger than any currently in use, then waits until it holds the smallest number. Ties are broken by process ID.

It satisfies all three critical section requirements for N processes but has the same practical limitations as Peterson's on modern hardware. It's important theoretically because it shows software-only mutual exclusion is possible for arbitrary N .

3.4 Semaphores

Software algorithms are elegant but fragile. In practice, we use hardware-supported primitives called **semaphores**. Introduced by Dijkstra in 1965, a semaphore is an integer variable accessed only through two atomic operations.

Semaphore: wait() and signal() operations

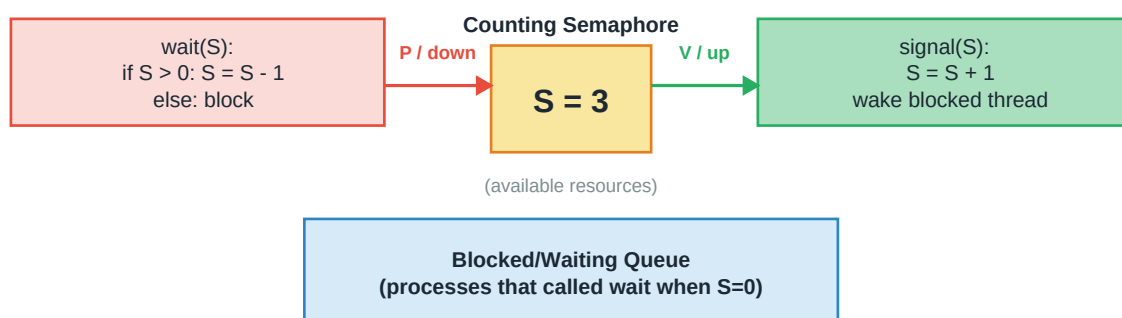


Figure: Semaphore — *wait()* decrements or blocks; *signal()* increments or wakes a waiter

```
wait(S) { signal(S) {
while (S <= 0) ; S = S + 1;
S = S - 1; }
}
```

The key word is **atomic** — the entire `wait()` or `signal()` operation happens as an indivisible unit. The hardware provides special instructions (like **test-and-set** or **compare-and-swap**) that the OS uses to implement these atomically.

The naive implementation shown above uses busy waiting — the process spins in a loop until the semaphore is positive. Real implementations put waiting processes to sleep in a queue and wake one up on `signal()`. This avoids wasting CPU.

Two Types of Semaphores

- **Binary Semaphore (Mutex):** Value is only 0 or 1. Perfect for mutual exclusion — initialize to 1, call `wait()` before critical section, `signal()` after.
- **Counting Semaphore:** Value can be any non-negative integer. Used to track how many instances of a resource are available. If you have 5 printers, initialize $S = 5$. Each `wait()` takes a printer; each `signal()` releases one.

Mutex vs Semaphore

The distinction between a mutex and a binary semaphore is subtle but important. A **mutex** is specifically a locking mechanism — the same thread that locks it must also unlock it (ownership). A **binary semaphore** is a signaling mechanism — any thread can call `signal()`, regardless of who called `wait()`. Semaphores are more flexible; mutexes are safer for pure mutual exclusion.

Feature	Mutex	Semaphore
Purpose	Locking (exclusion)	Signaling / counting
Ownership	Only owner can unlock	Anyone can signal
Value	Boolean (locked/unlocked)	Integer (0..N)
Priority inversion handling	Often built-in	Not by default
Use case	Protect shared data	Coordinate events, count resources

3.5 Monitors and Priority Inversion

Monitors

Semaphores are powerful but error-prone — forgetting a `signal()` or getting the order of `wait()`s wrong leads to deadlocks and bugs that are nearly impossible to reproduce. **Monitors** are a higher-level synchronization construct that encapsulate shared data and the operations on it, guaranteeing that only one process is inside the monitor at a time.

Inside a monitor, you can use **condition variables** to wait for specific events. A condition variable has two operations: `wait()` (releases the monitor and blocks) and `signal()` (wakes one waiter). Java's `synchronized` methods and `wait/notify` are essentially monitors.

Priority Inversion

Priority inversion is a subtle bug that shows up in priority-based systems. It goes like this: A low-priority process L holds a lock. A high-priority process H wants the lock, so H blocks. Meanwhile, a medium-priority process M arrives — but M doesn't need the lock. The scheduler prefers M over L (higher priority), so M runs, L doesn't run, L can't release the lock, H stays blocked. Result: medium-priority M is effectively higher priority than H.

Important: Priority inversion nearly killed the Mars Pathfinder mission in 1997. The rover kept resetting because a high-priority task was blocked on a semaphore held by a low-priority task that never got scheduled. The fix was **priority inheritance**: while L holds the lock H needs, L temporarily inherits H's priority so it can run and release the lock.

3.6 Classical IPC Problems

These three problems appear in every OS textbook because they capture common synchronization patterns. Understanding them means understanding synchronization deeply.

Producer-Consumer (Bounded Buffer)

One or more processes (producers) generate data items and place them in a shared buffer. Other processes (consumers) remove items and process them. The buffer has finite capacity. We need to ensure: producers wait when buffer is full, consumers wait when buffer is empty, and only one process modifies the buffer at a time.

Producer-Consumer Problem (Bounded Buffer)

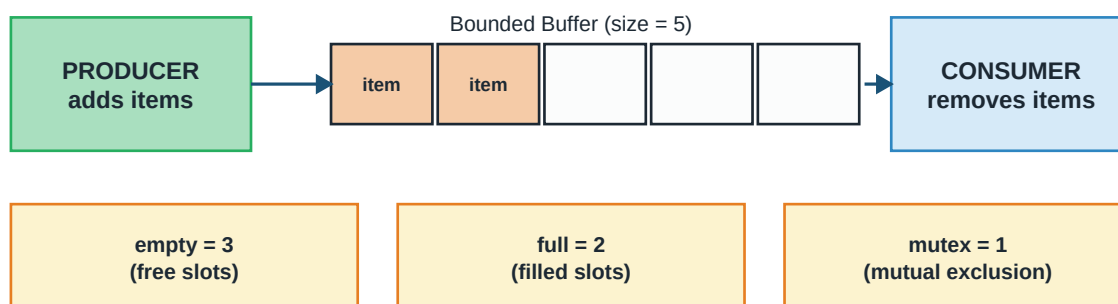


Figure: Producer-consumer — three semaphores coordinate buffer access

The solution uses three semaphores:

- `empty` — counts free slots (initially N). Producer waits on this.
- `full` — counts filled slots (initially 0). Consumer waits on this.
- `mutex` — binary, protects the actual buffer manipulation.

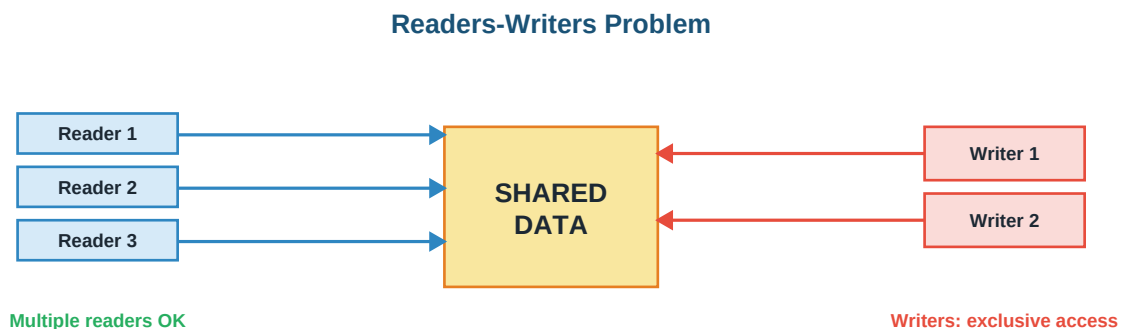
```
// Producer // Consumer
wait(empty); wait(full);
wait(mutex); wait(mutex);
add item to buffer; remove item from buffer;
```

```
signal(mutex); signal(mutex);  
signal(full); signal(empty);
```

Key Insight: The order matters! You must wait on the counting semaphore BEFORE the mutex, otherwise you can deadlock. If a producer grabs mutex and then finds empty=0, it holds the mutex forever, blocking consumers too.

Readers-Writers Problem

Multiple processes want to access shared data. Some only read (readers), some modify (writers). Multiple readers can safely access the data at once, but writers need exclusive access. How do we coordinate this efficiently?



Multiple readers can read simultaneously, but writers need exclusive access

Figure: Readers-writers — readers share, writers get exclusive access

There are two classic variants with different fairness policies. The **first readers-writers problem** gives priority to readers: no reader waits unless a writer has already gained access. This can starve writers if readers keep arriving. The **second readers-writers problem** gives priority to writers: once a writer is waiting, no new readers may start. This can starve readers. Real systems typically implement fair variants that alternate or use FIFO ordering.

Dining Philosophers

Five philosophers sit around a circular table. Between each pair is one chopstick — five total. Each philosopher alternates between thinking and eating. To eat, a philosopher needs BOTH the chopstick to their left AND the chopstick to their right. How do we let philosophers eat without deadlock or starvation?

Dining Philosophers Problem

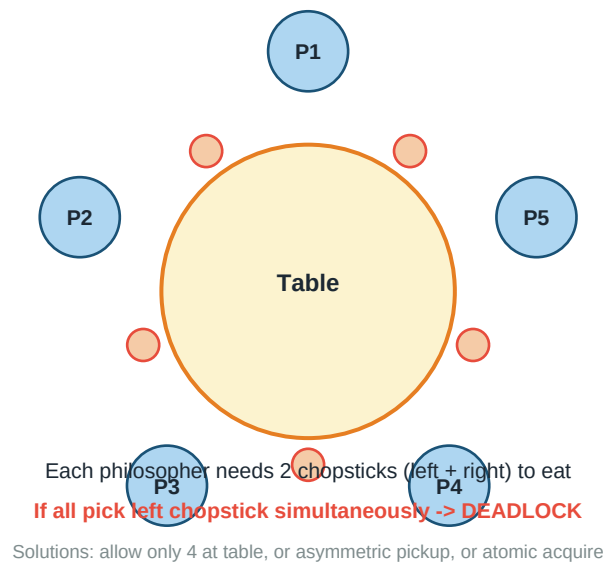


Figure: Dining philosophers — a small problem that captures many big issues

The naive solution — each philosopher picks up left chopstick, then right — deadlocks if all five pick up their left chopstick simultaneously. Nobody has both chopsticks, nobody puts one down, everyone waits forever.

Classic fixes:

- **Allow at most 4 philosophers** at the table. At least one philosopher can always get both chopsticks.
- **Asymmetric solution:** Odd-numbered philosophers pick up left first, even ones pick up right first. Breaks the symmetry that causes circular wait.
- **Atomic acquisition:** A philosopher only picks up chopsticks when both are available, and does so atomically.

Key Insight: Dining philosophers is not just a puzzle — it models any situation where multiple processes need multiple shared resources. Every time you take two locks in a specific order to avoid deadlock, you're solving a dining philosophers instance.

Chapter 4

Deadlock

Sometimes synchronization goes wrong not because of a race but because everyone is politely waiting for someone else. This chapter explains what deadlock is, when it happens, and the four strategies for dealing with it.

4.1 What is Deadlock?

A **deadlock** is a state where a set of processes are all waiting for resources held by other processes in the set, so none can proceed. Unlike starvation (which might eventually resolve), deadlock is permanent — without outside intervention, the processes will wait forever.

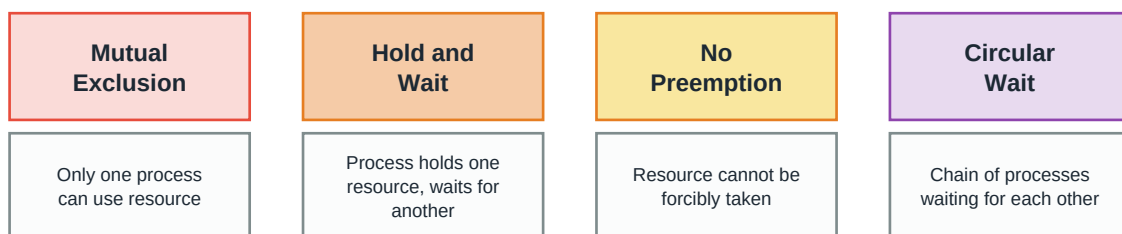
Analogy: Two cars arrive at a narrow one-lane bridge from opposite ends. Each driver waits for the other to back up. Neither will. That's deadlock — a stable state of mutual waiting where nothing will happen without external intervention (a traffic cop deciding who backs up).

Example: A concrete example: Process P1 holds a lock on file A and wants to acquire a lock on file B. Process P2 holds a lock on file B and wants a lock on file A. Neither can proceed. Neither will release its lock. Both processes are frozen forever.

4.2 The Four Necessary Conditions

Coffman, Elphick, and Shoshani proved in 1971 that deadlock can only occur when **all four** of the following conditions hold simultaneously. This isn't a coincidence — each condition is essential, and breaking any one prevents deadlock.

Four Necessary Conditions for Deadlock (ALL must hold)



Break ANY one condition to prevent deadlock

Figure: Four conditions — all must hold for deadlock; break any one to prevent it

1. Mutual Exclusion

At least one resource must be non-sharable — only one process can use it at a time. If everything were shareable, no process would ever have to wait. Some resources are inherently non-shareable (a printer can't print two documents at once) so we can't always eliminate this.

2. Hold and Wait

A process holding at least one resource is waiting to acquire additional resources held by other processes. If processes always requested all resources at once, or never held any while waiting, this wouldn't happen.

3. No Preemption

Resources cannot be forcibly taken away — a process must release them voluntarily. If the OS could just yank a resource back and give it to someone else, we could break the cycle.

4. Circular Wait

There must be a chain of processes $P_1, P_2, \dots, P_n$ such that P_1 waits for a resource held by P_2 , P_2 waits for one held by P_3 , ..., and P_n waits for one held by P_1 . Without the circle, someone in the chain can eventually proceed and release resources for others.

Important: All four conditions are NECESSARY but not SUFFICIENT. Even if all four hold, deadlock may not happen immediately — the exact resource requests must line up in a bad way. But once deadlock happens, all four conditions must have held.

4.3 The Resource Allocation Graph (RAG)

The RAG is a visual tool for reasoning about deadlock. It's a directed graph with two types of nodes: **processes** (drawn as circles) and **resource types** (drawn as rectangles, with dots inside for individual instances). Two types of edges:

- **Request edge ($P_i \rightarrow R_j$):** Process P_i has requested resource R_j but not yet been granted it.
- **Assignment edge ($R_j \rightarrow P_i$):** Resource R_j has been allocated to process P_i .

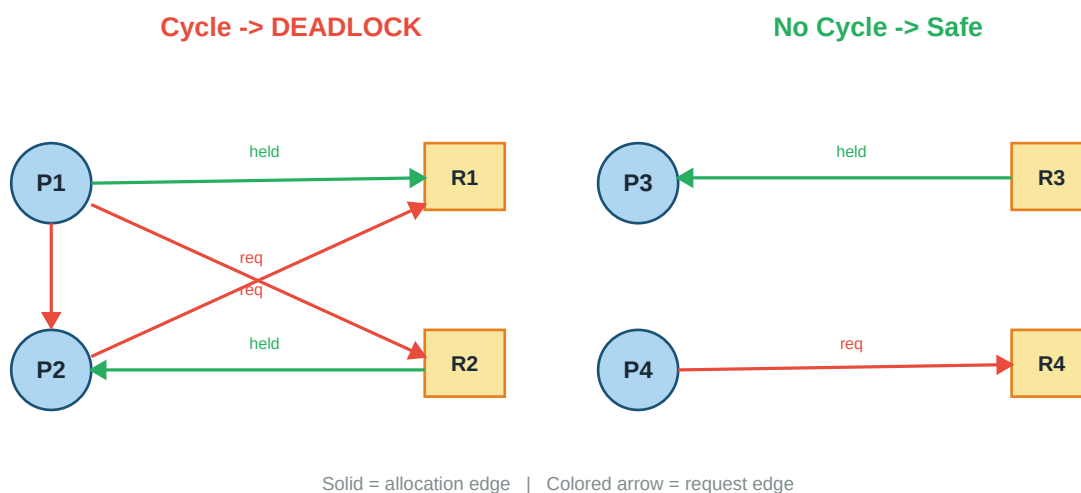


Figure: Left: cycle in RAG = deadlock. Right: no cycle = no deadlock

The theorem: if the graph has no cycle, there is no deadlock. If it has a cycle: (a) with only one instance of each resource type, there IS deadlock. (b) with multiple instances, deadlock is possible but not guaranteed —

you have to analyze further.

4.4 Four Strategies for Handling Deadlock

Approaches to Deadlock Handling



Trade-off: prevention wastes resources; ignoring risks system hangs

Figure: Four approaches — trade-offs among safety, cost, and complexity

Strategy 1: Prevention

Design the system so that at least one of the four necessary conditions cannot hold. If we prevent circular wait, deadlock is impossible.

- **Break mutual exclusion:** Make resources shareable. Only works for inherently shareable resources (like read-only files).
- **Break hold and wait:** Require processes to request all resources upfront before starting. Simple but wasteful — a process might hold resources it doesn't use yet.
- **Break no preemption:** If a process requests a resource that can't be immediately allocated, force it to release everything it holds. Try again later. Only works for resources whose state can be saved and restored, like CPU registers.
- **Break circular wait:** Impose a total ordering on resource types. Processes must request resources in strictly increasing order. This is the most practical prevention technique.

Strategy 2: Avoidance

Instead of preventing conditions, allow them but never enter an unsafe state. This requires knowing each process's maximum resource needs in advance.

Banker's Algorithm

Named after how a bank manages loans — never grant a loan that would leave the bank unable to satisfy any customer's maximum need. The Banker's Algorithm:

- Each process declares its maximum need for each resource type.
- When a process requests resources, the OS pretends to grant it.
- Then checks: can we still find a sequence in which every process gets its maximum, completes, and releases resources for the next?
- If yes → safe state → grant the request. If no → unsafe state → make the process wait.

Data structures:

Available[m] - free instances of each resource type
Max[n][m] - maximum need of each process
Allocation[n][m] - resources currently held
Need[n][m] - Max - Allocation

Safety algorithm — find a safe sequence:

- **1.** Work = Available; Finish[i] = false for all i.
- **2.** Find i such that Finish[i] = false AND Need[i] ≤ Work.
- **3.** Work = Work + Allocation[i]; Finish[i] = true; go to step 2.
- **4.** If Finish[i] = true for all i → state is safe.

Important: The Banker's Algorithm requires knowing maximum resource needs in advance, which is often impractical. It's rarely used in real OSes but is a classic teaching example.

Strategy 3: Detection and Recovery

Let deadlocks happen, detect them, then recover. The detection algorithm is similar to Banker's — look for a completion sequence. If none exists, deadlock is present.

Recovery options:

- **Process termination:** Kill deadlocked processes. Options: kill all of them at once (drastic), or kill one at a time until deadlock breaks (rechecking each time).
- **Resource preemption:** Forcibly reclaim resources from some processes and give them to others. Requires rollback of the preempted processes, and careful selection to avoid starvation.

Strategy 4: Ignore the Problem (Ostrich Algorithm)

Named after the myth that ostriches bury their heads in sand — just pretend deadlock doesn't happen. Surprisingly, this is what most general-purpose OSes (Linux, Windows, macOS) do! Their reasoning: deadlocks are rare, and the cost of detection or prevention outweighs the occasional need to reboot. For a desktop system, this is acceptable. For a life-critical system, it isn't.

Key Insight: The choice of strategy depends on how often deadlocks occur, how critical avoidance is, and how much runtime overhead you can afford. Real systems mix strategies — for example, using ordering to prevent certain deadlocks while ignoring the rest.

Chapter 5

Multithreading

A process is expensive — its own address space, its own file table, its own everything. Threads are a lighter-weight way to get concurrency, sharing everything except the tiny amount of state that makes them independent. This chapter is about threads and how the OS supports them.

5.1 Threads — Lightweight Processes

So far we've talked about processes as the units of concurrent execution. But processes have a lot of overhead. Creating one is expensive (new address space, new page tables, copy of parent's memory). Communicating between them requires special IPC mechanisms because they don't share memory. Context switching is slow.

A **thread** is a smaller unit of execution within a process. Multiple threads inside the same process share the code, data, heap, and open files, but each has its own program counter, registers, and stack. Threads are cheap to create, cheap to switch, and can communicate directly through shared memory.

Analogy: A process is a house. A thread is a person living in it. Multiple people can live in the same house — sharing the kitchen (data), the wifi (files), and the front door (address space). But each person has their own bed (stack) and their own current activity (program counter and registers). Adding a person is cheap; building a new house is expensive.

Process (single thread) vs Multi-threaded Process

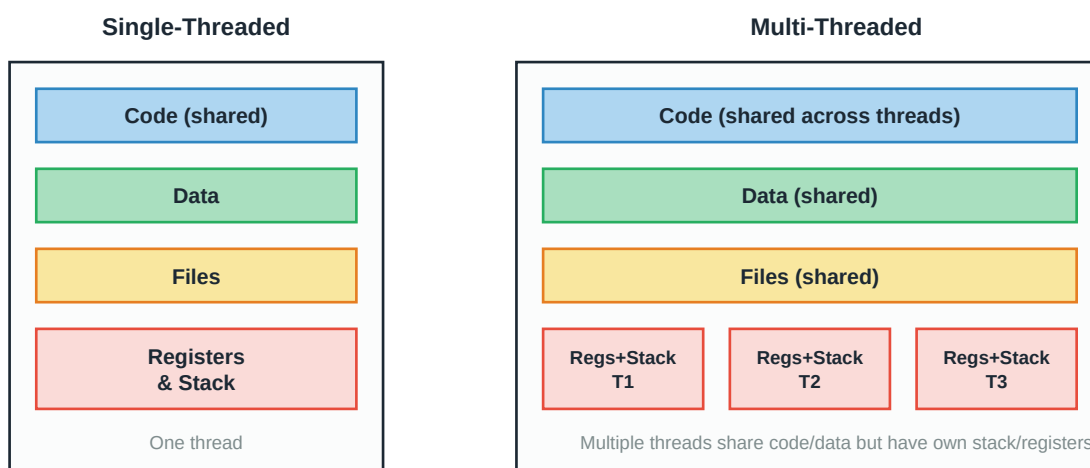


Figure: Single-threaded process vs multi-threaded — threads share most of the process's state

Why Use Threads?

- **Responsiveness:** A word processor can spell-check in one thread while the user types in another. The UI stays responsive even during long operations.

- **Resource sharing:** Threads share memory by default, so passing data between them is a matter of reading and writing shared variables — no IPC required.
- **Economy:** Creating a thread is about 100x cheaper than creating a process. Context switching between threads of the same process is much faster because you don't have to swap address spaces.
- **Scalability:** On a multi-core CPU, multiple threads can execute truly in parallel — one thread per core. A single-threaded process can only use one core no matter how many are available.

Process vs Thread — A Direct Comparison

Aspect	Process	Thread
Address space	Separate	Shared with sibling threads
Creation cost	High (new page tables, copy memory)	Low (just a stack + registers)
Context switch cost	High (address space change)	Low (same address space)
Communication	Explicit IPC (pipes, sockets)	Shared memory directly
Failure isolation	Strong (crash affects only self)	Weak (crash may kill whole process)
Synchronization needed?	Only when sharing IPC channels	Yes, for all shared data

Important: The flip side of easy sharing is easy corruption. Threads must synchronize (with mutexes, semaphores, etc.) whenever they access shared data — the race conditions from Chapter 3 apply. This is where most threading bugs come from.

5.2 User-Level vs Kernel-Level Threads

Threads can be implemented in two very different ways, with a fundamental trade-off between speed and the kernel's awareness.

User-Level Threads (ULT)

A user-space thread library (like an old POSIX library or Java's original green threads) manages threads entirely within the process. The kernel sees only one process and knows nothing about the threads inside. Thread creation, scheduling, and context switching all happen in user space with no kernel calls.

Advantages: Extremely fast — no kernel involvement means no mode switches. Portable across different kernels. Customizable scheduling policies.

Disadvantages: If any thread makes a blocking system call, the ENTIRE process blocks — even if other threads have work to do. The kernel can't run threads in parallel on multiple CPUs because it only sees one schedulable entity per process.

Kernel-Level Threads (KLT)

The kernel is aware of each thread individually. Thread creation and scheduling are done by the kernel. Threads of the same process can run on different CPUs in parallel. If one thread blocks, others in the same process can continue.

Advantages: True parallelism on multi-core systems. Blocking one thread doesn't block others. The kernel can make smart scheduling decisions across threads.

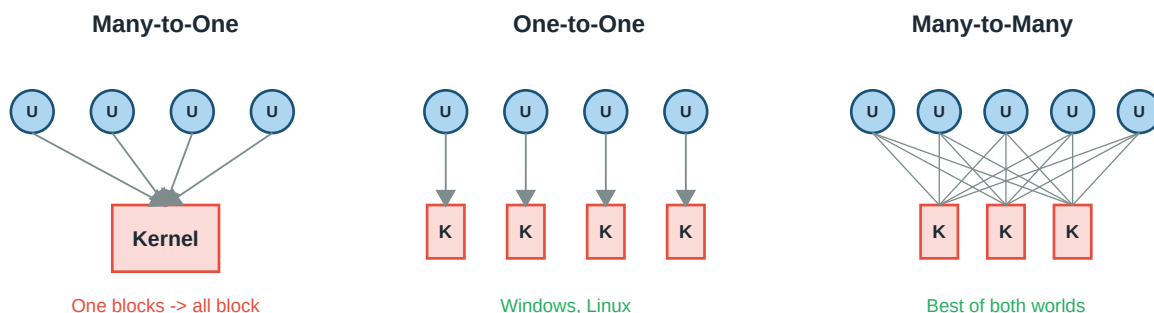
Disadvantages: Every thread operation (create, destroy, switch) requires a system call, which is slower than user-space operations. Uses kernel resources per thread, so there's a practical limit on thread count.

Feature	User-Level Threads	Kernel-Level Threads
Managed by	User library	Kernel
OS awareness	OS sees only the process	OS sees each thread
Context switch	Fast (no kernel involvement)	Slower (requires mode switch)
Blocking calls	Blocks whole process	Only that thread blocks
Multi-core parallelism	No (all on one core)	Yes
Examples	GNU Portable Threads, early Java	Windows threads, Linux NPTL

5.3 Multithreading Models

Modern systems often use a mix of user and kernel threads. The question is: how do user threads map to kernel threads? There are three standard patterns.

Multithreading Models



Legend: U = user thread, K = kernel thread

OS is aware of and schedules kernel threads; user threads managed by library

Figure: Three threading models — different ways to map user threads to kernel threads

Many-to-One Model

Many user threads mapped to a single kernel thread. Thread management is done in user space, so it's efficient. But since the kernel sees only one thread, if any user thread blocks on a system call, ALL user threads block. This model can't take advantage of multi-core CPUs. Old systems used this; almost no modern system does.

One-to-One Model

Each user thread maps to a single kernel thread. This solves the blocking problem — one thread blocking doesn't affect others — and allows real parallelism across cores. But creating a user thread requires creating a kernel thread, so it's more expensive. This is what Linux (NPTL) and Windows use today.

Many-to-Many Model

Many user threads multiplexed onto a smaller-or-equal number of kernel threads. Combines advantages of both: the developer can create as many user threads as needed without worrying about kernel resources, and the kernel can run multiple threads in parallel. Implementation is complex; some older systems (like Solaris in early versions) used this, but modern systems have mostly moved to one-to-one because kernel threads have become cheap enough.

A Related Concept: Two-Level Model

A variant of many-to-many where a specific user thread can be **bound** to a specific kernel thread. Bound threads get guaranteed CPU time; unbound threads share the remaining kernel threads. Used for threads that must not be interrupted (like a real-time task).

5.4 Benefits of Multithreading — Revisited

We've seen the advantages abstractly. Let's ground them in concrete scenarios you'll see in real software.

1. Web Servers

A web server needs to handle many client requests concurrently. Creating a new process per request would be prohibitively expensive. Instead, servers like Apache or Nginx use a thread pool — a fixed number of threads pulling requests from a queue. Each request gets a thread; the thread completes the request and returns to the pool.

2. UI Applications

Every GUI has an event loop running in a main thread. Long operations (file I/O, network calls) must run in background threads or the UI freezes. When Windows shows "Not Responding", it means the main thread hasn't returned to the event loop within a reasonable time.

3. Parallel Computation

A scientific computation that takes 60 seconds on one core can theoretically take 15 seconds on four cores, if you can split the work into four independent threads. This is embarrassingly parallel work — computations like image processing where each pixel can be handled independently.

Key Insight: Amdahl's Law is worth knowing: if a program is 90% parallelizable, the maximum speedup on infinite cores is only 10x — because the 10% serial part becomes the bottleneck. Real-world speedups are usually much less than the naive "N cores → N times faster" would suggest.

Chapter 6

Memory Management

Every process needs memory. But how does the OS give memory to many processes at once, keep them from stepping on each other, and let programs use more memory than physically exists? This is the biggest chapter for a reason — memory management is the OS's most sophisticated job.

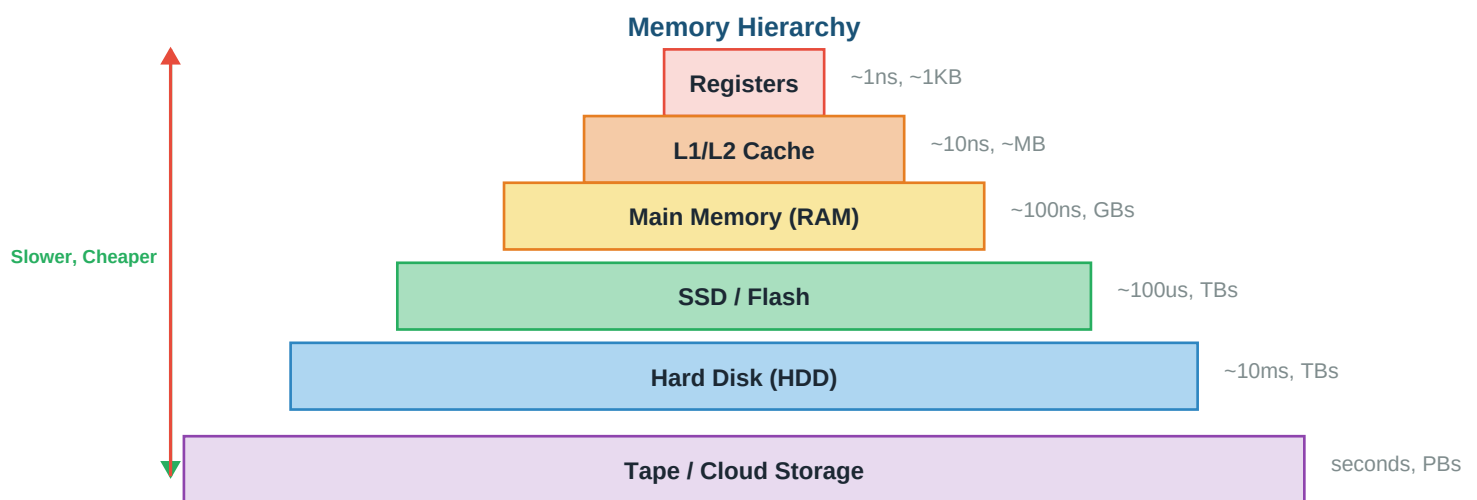
6.1 The Memory Management Problem

Memory management sits at the center of the OS's job. Every process needs memory: for its code, its data, its stack, and its dynamically allocated objects. The OS must decide who gets what memory, prevent processes from reading or modifying each other's memory, use memory efficiently, and hide the complexity from programmers.

The challenges compound: physical memory is limited. Processes come and go, leaving fragmented free space. Different processes have different needs — some tiny, some huge. And the CPU must translate every memory access, so any scheme has to be fast enough to happen billions of times per second.

Memory Hierarchy

Before we dive into how the OS manages memory, understand the memory hierarchy. Faster memory is expensive per byte; cheaper memory is slower. Systems combine multiple levels to get the best of both.



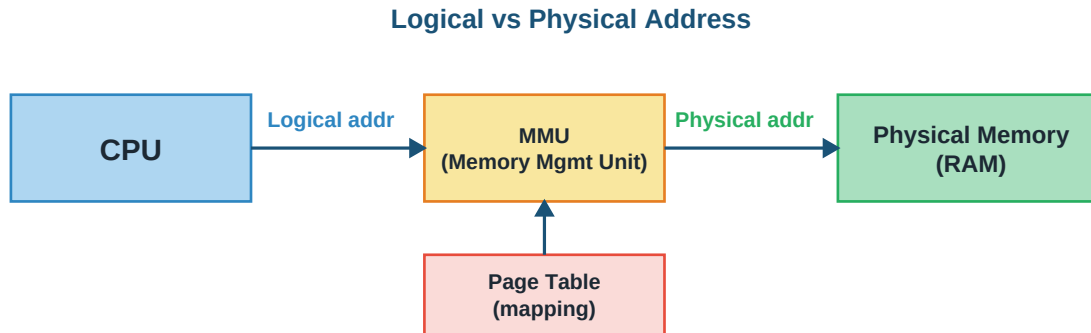
As we go down: capacity increases, cost per byte and access time increase

Figure: Memory hierarchy — faster near the top, larger near the bottom

The huge speed gap between RAM (nanoseconds) and disk (milliseconds) — a factor of 100,000 — is the reason virtual memory works. Most programs don't need all their data at once, so keeping only the active parts in RAM and the rest on disk works well most of the time.

6.2 Logical vs Physical Addresses

When your program says `int x = 42;`, the CPU stores 42 somewhere — but where? Not directly at some fixed RAM address. Modern systems use **address translation**: the CPU generates a **logical address** (what the program sees) and hardware translates it to a **physical address** (actual RAM location).



MMU is hardware; page table lookup is fast (with TLB cache)

Figure: Logical to physical address translation via the MMU

- **Logical (virtual) address:** Generated by the CPU during instruction execution. Programs see only these. Each process has its own logical address space, starting from 0.
- **Physical address:** Actual location in RAM. There's only one physical address space, shared by all processes.
- **Memory Management Unit (MMU):** A piece of hardware inside the CPU that translates logical to physical on every memory access. Uses tables (page tables or segment tables) that the OS maintains.

Key Insight: This separation is powerful. Because each process has its own logical space, processes can't accidentally or maliciously access each other's memory. Because the OS controls the translation, it can move a process's memory around physically without the process noticing. And because the physical space is hidden, programs can behave as if they had more memory than actually exists.

6.3 Contiguous Memory Allocation

The oldest and simplest memory scheme: give each process a single contiguous chunk of physical memory. It's easy to implement — one base register (start address) and one limit register (size) per process, and the MMU does $\text{physical} = \text{logical} + \text{base}$ after checking $\text{logical} < \text{limit}$.

Fixed Partitioning

Divide memory into fixed-size partitions at boot time. Each partition holds one process. Simple to manage — just a bitmap of free partitions. But the partitions are set — if you get a job that's smaller than the partition, the leftover space in that partition is wasted.

Variable (Dynamic) Partitioning

Give each process exactly the amount of memory it needs. The OS keeps a list of free "holes". When a process arrives, find a hole big enough. When the process terminates, mark its memory as free (possibly merging with adjacent free holes). More flexible than fixed partitioning.

The Fragmentation Problem

Both schemes lead to **fragmentation** — wasted memory that we can't use even though it's technically free.

Internal vs External Fragmentation



Figure: Internal fragmentation (waste inside a partition) vs external (scattered free space)

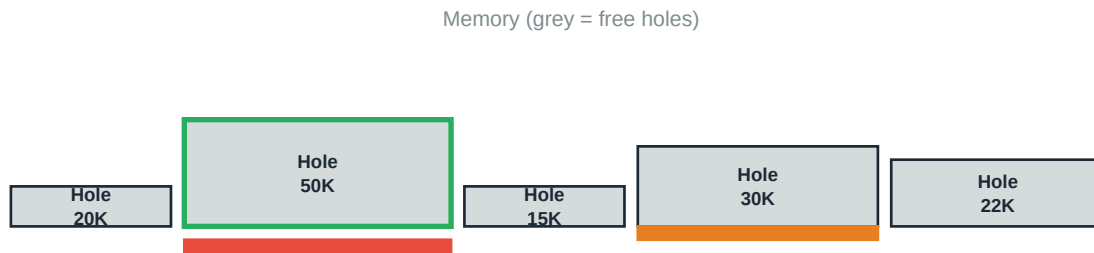
- **Internal fragmentation:** Space wasted **INSIDE** an allocated block. If a process needs 60KB but is given a 100KB partition, 40KB is wasted internally. This is a fixed-partitioning problem.
- **External fragmentation:** Free space exists but is scattered in pieces too small to satisfy any request. Total free memory might be 45KB, but if it's in three 15KB chunks, we can't allocate a 40KB request. This is a variable-partitioning problem.

Compaction can fix external fragmentation — shuffle all processes together to create one big free hole. But it's expensive: you have to pause execution, copy memory around, and update every reference. Modern systems avoid compaction by using paging (below).

Allocation Strategies for Variable Partitioning

When a request arrives, which hole do we pick?

Allocating a 25 KB request - Which hole?



First Fit -> picks 50K

Best Fit -> picks 30K

Worst Fit -> picks 50K (leaves 25K)

Figure: First-fit, best-fit, worst-fit — different choices for the same request

- **First Fit:** Allocate the FIRST hole big enough. Fastest search. Tends to leave small unusable fragments near the start of memory.
- **Best Fit:** Allocate the SMALLEST hole that fits. Produces the smallest leftover fragment — but those tiny leftovers pile up and become unusable. Slow because it must search the whole free list.
- **Worst Fit:** Allocate the LARGEST hole. The idea: the leftover space stays big enough to satisfy another request. In practice, this is usually the worst-performing of the three.
- **Next Fit:** Like first fit, but starts searching from where the last allocation ended. Spreads allocations more evenly.

Buddy System

A clever compromise. Memory is allocated in sizes that are powers of 2. To allocate 100 bytes, round up to 128. If a 128-byte block is free, allocate it. If not, split a 256-byte block into two 128-byte "buddies" and allocate one. When a block is freed, check if its buddy is also free — if so, merge them into the larger block. Simple to implement, fast, but suffers from internal fragmentation (up to 50% in the worst case).

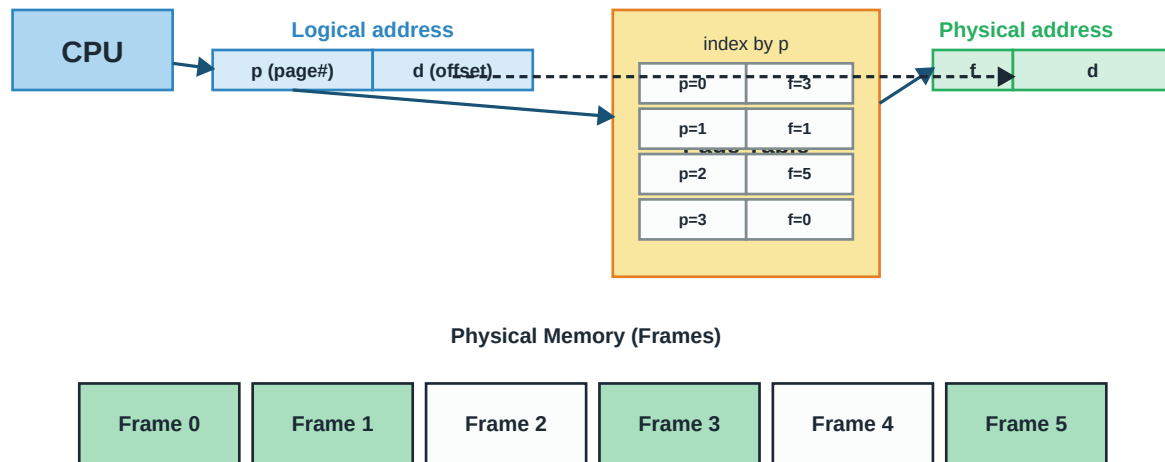
The Linux kernel uses a buddy allocator for physical page frames.

6.4 Paging

Paging is the killer idea of memory management. Instead of allocating each process a contiguous chunk, we divide memory into fixed-size pieces and let a process's memory be scattered anywhere. This completely eliminates external fragmentation.

Physical memory is divided into fixed-size **frames** (typically 4KB). A process's logical address space is divided into **pages** of the same size. The OS maintains a **page table** that maps each page to the frame currently holding it. Pages of the same process can be scattered anywhere in physical memory.

Paging: Logical to Physical Address Translation



Page 2 -> Frame 5; Page 3 -> Frame 0; Page 0 -> Frame 3

Pages of same process can be scattered across non-contiguous frames

Figure: Paging — logical addresses are (page, offset); page table maps page to frame

How Address Translation Works

The CPU generates a logical address, which the MMU splits into two parts: the high-order bits are the **page number (p)**; the low-order bits are the **offset (d)** within the page. The MMU uses p to index into the page table, reads the corresponding frame number f, and constructs the physical address by concatenating f with d.

Logical address: [page number p | offset d]

Page table lookup: p -> f

Physical address: [frame number f | offset d]

Example: Assume 32-bit logical addresses, 4KB pages. Then offset is 12 bits ($\log_2 4096 = 12$) and page number is 20 bits — so there are $2^{20} \approx 1$ million pages per process. Each page table entry is typically 4 bytes, so the full page table is 4MB per process. That's a lot — multi-level page tables address this.

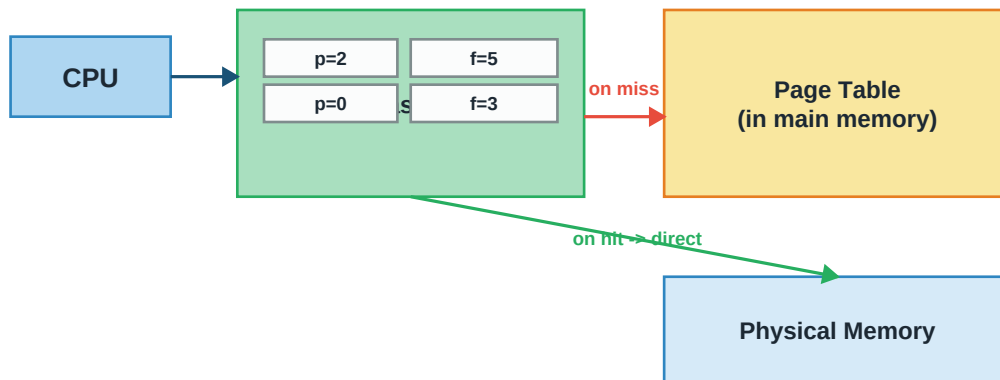
Key Insight: Paging eliminates external fragmentation entirely: any page can go in any frame. It may still have small internal fragmentation in the last page of a process (if the process isn't an exact multiple of page size). But that's at most 1 page = 4KB per process — negligible.

The TLB — Making Paging Fast

There's a problem with paging as described: every memory access now requires TWO memory accesses — one to read the page table, one to read the actual data. That's a 2x slowdown, unacceptable in practice.

The solution: cache the recent translations in a very small, very fast hardware cache called the **Translation Lookaside Buffer (TLB)**. The MMU checks the TLB first. If the page number is there (a TLB hit), we get the frame number instantly. If not (a TLB miss), we walk the page table in memory and update the TLB.

TLB: Fast Cache for Page Table



$$EAT = h(t+m) + (1-h)(t+2m), \text{ where } h=\text{hit ratio, } t=\text{TLB time, } m=\text{memory time}$$

Figure: TLB caches page table entries — most accesses hit the TLB and skip the page table

Effective Access Time (EAT):

$$EAT = h(t + m) + (1-h)(t + 2m)$$

where h = TLB hit ratio, t = TLB access time, m = memory access time

Example: If TLB access is 20ns, memory is 100ns, and hit ratio is 98%:

$$EAT = 0.98(20 + 100) + 0.02(20 + 200) = 117.6 + 4.4 = 122\text{ns.}$$

Compare to unpaged 100ns — the overhead is only 22%, thanks to the TLB.

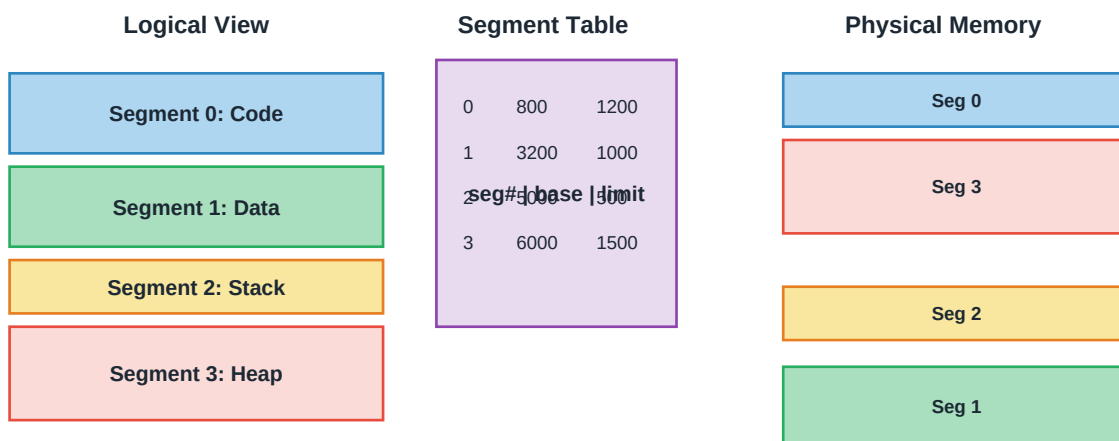
Important: Every context switch potentially invalidates the TLB, because the new process has a different page table. This is one reason context switches are expensive. Modern CPUs use process-tagged TLBs (each entry stores a process ID) to avoid full flushes.

6.5 Segmentation

Paging is elegant but hides the logical structure of a program. From a paging perspective, code, data, and stack are just an undifferentiated stream of pages. But programmers think in terms of modules — code, globals, heap, stack, per-thread data. **Segmentation** reflects this view.

In segmentation, memory is divided into **variable-sized segments**, each corresponding to a logical unit: one segment for code, one for globals, one for the heap, one per procedure's local variables, and so on. Each segment has a name (or number) and a length. A logical address is a pair (segment number, offset within segment).

Segmentation: Variable-sized logical divisions



Address = <segment#, offset>; hardware checks offset < limit

Segments correspond to logical program units (user-visible)

Figure: Segmentation — variable-sized logical units mapped to physical memory

The **segment table** maps each segment to its base address in physical memory and its length. Address translation: look up segment s to get base and limit. Check offset < limit. Compute physical = base + offset.

Important: Segmentation solves paging's opacity but reintroduces external fragmentation because segments are variable-sized. Modern systems use **segmentation with paging**: each segment is broken into pages, combining the logical view of segmentation with the efficient allocation of paging. Intel's x86 architecture historically used this hybrid model, though modern OSes essentially ignore segments and rely purely on paging.

6.6 Virtual Memory

So far we've assumed every page of every running process is in RAM. But what if a program is larger than RAM? Or what if we want to run more processes than RAM can hold?

Virtual memory is the elegant answer: keep only the actively-used pages in RAM. Store the rest on disk (in a special area called **swap space** or the **page file**). When the program tries to access a page that isn't in RAM, transparently load it from disk. To the program, it looks like all the memory is there.

Virtual Memory: Program can be larger than RAM

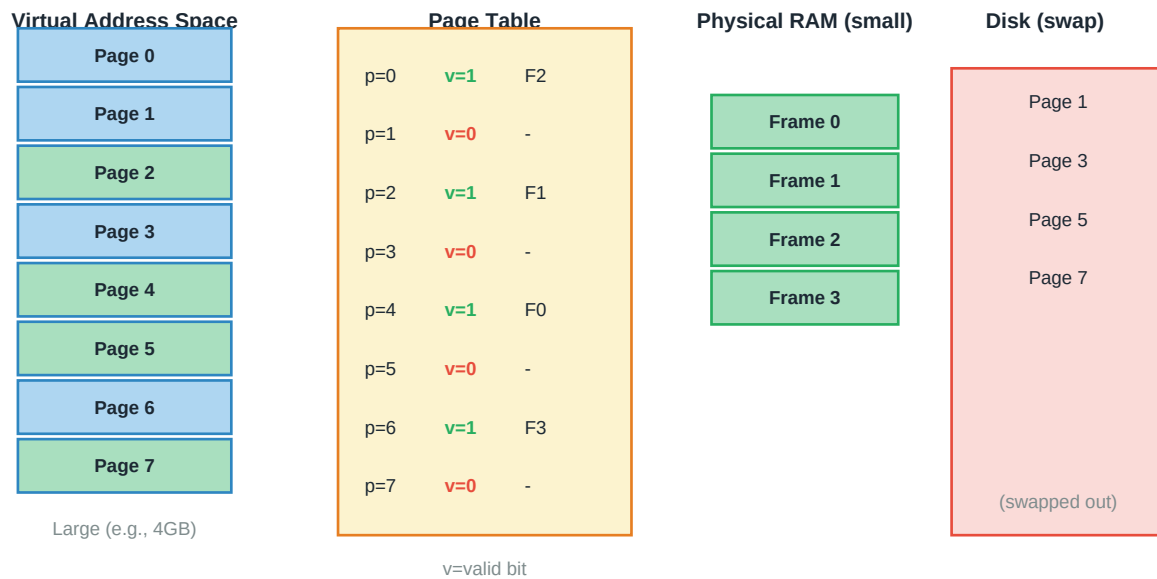


Figure: Virtual memory — pages live on disk when not needed in RAM

Analogy: Think of a big library. You don't put every book on your desk — just the ones you're currently reading. When you need a book, you walk to the shelf and fetch it. The desk (RAM) can be small even though your library (virtual memory) is huge. Virtual memory works because programs, like readers, tend to focus on a small working set at any given time.

Demand Paging

Demand paging is the strategy: only load pages when they're demanded (referenced), not in advance. Initially, none of a program's pages are in RAM. As it runs and accesses memory, each new access to an absent page triggers a **page fault**.

Handling a Page Fault

Page Fault Handling Steps

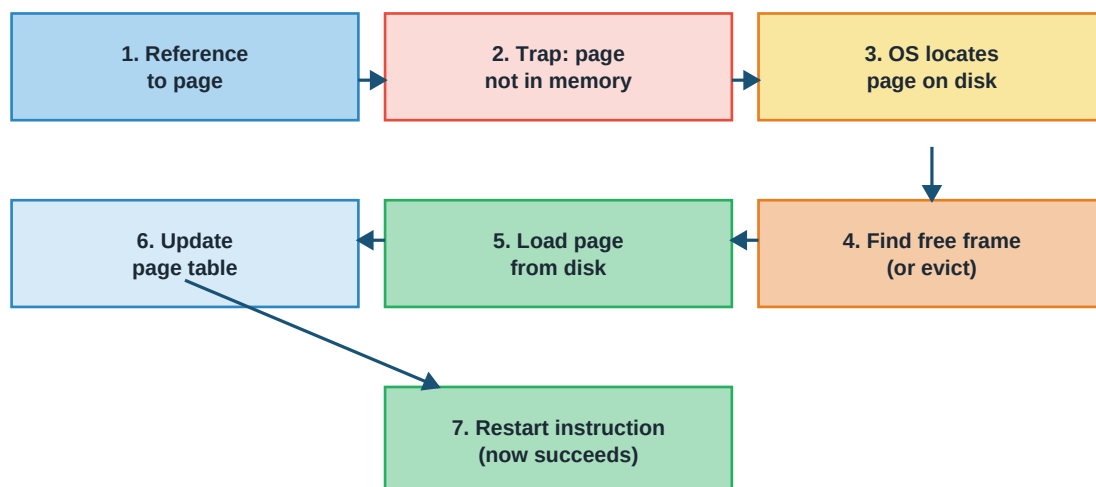


Figure: Page fault handling — trap, locate, load, restart

- **1.** Process references a page. MMU consults page table. Valid bit is 0.
- **2.** MMU triggers a **page fault** — a trap to the OS.
- **3.** OS checks: is this a legal reference? (If not, kill the process.) If yes, locate the page in swap space.
- **4.** OS finds a free frame in RAM. If no free frame, use page replacement (Section 6.7) to evict one.
- **5.** OS reads the page from disk into the frame — this is the slow part, milliseconds.
- **6.** OS updates the page table: valid bit = 1, frame number filled in.
- **7.** OS restarts the instruction that caused the fault. This time it succeeds.

EAT (with page faults):

$EAT = (1-p) \times ma + p \times \text{page_fault_service_time}$

where p = page fault rate, ma = memory access time

Example: If memory access is 100ns and page fault service is 10ms (10,000,000ns):

At $p = 0.001$ (0.1% faults): $EAT = 0.999(100) + 0.001(10,000,000) = 10,100\text{ns}$ — 100x slower!

At $p = 0.00001$ (very rare faults): $EAT = 200\text{ns}$ — 2x slower.

This tells you: page faults must be RARE for virtual memory to be fast.

Advantages of Virtual Memory

- **Programs can be larger than RAM.** The OS pages them in and out as needed.
- **Higher degree of multiprogramming.** More processes fit because each only keeps its working set in RAM.
- **Faster startup.** A large program starts running before all its pages are loaded — only the initial pages are needed.
- **Simpler programming.** No need to manually manage overlays or memory bank switching (the way old systems worked).

6.7 Page Replacement Algorithms

When a page fault happens and RAM is full, we must evict some existing page to make room. Which one? This is the **page replacement** problem. The goal is to minimize future page faults.

Reference Strings

To evaluate algorithms, we use a **reference string** — a sequence of page numbers a process accesses. Given the reference string and the number of frames, we can trace through each algorithm and count faults.

Page Replacement Algorithms (3 frames, string: 1,2,3,4,1,2,5,1,2,3,4,5)

	1	2	3	4	1	2	5	1	2	3	4	5
FIFO	1	1	1	4	4	4	5	5	5	5	5	5
	-	2	2	2	1	1	1	1	1	3	3	3
	-	-	3	3	3	2	2	2	2	2	4	4
LRU	1	1	1	4	4	4	5	5	5	5	5	5
	-	2	2	2	1	1	1	1	1	3	3	3
	-	-	3	3	3	2	2	2	2	2	4	4
OPT	1	1	1	1	1	1	1	1	1	3	4	4
	-	2	2	2	2	2	2	2	2	2	2	2
	-	-	3	4	4	4	5	5	5	5	5	5

FIFO: 9 faults | LRU: 9 faults | Optimal: 7 faults

Red = page fault (miss) | Green = hit

Figure: Three algorithms on the same reference string — Optimal wins, LRU comes close

1. First-In First-Out (FIFO)

Evict the page that has been in memory longest. Simple to implement — just a queue. But it doesn't consider whether the oldest page is still being used, so it can evict a heavily-used page. Suffers from **Belady's anomaly** (see below).

2. Optimal (OPT / Belady's Algorithm)

Evict the page that will NOT be used for the longest time in the future. This provably minimizes page faults for any reference string. But it requires knowing the future, which is impossible. Optimal is used as a theoretical benchmark to see how close real algorithms get.

3. Least Recently Used (LRU)

Evict the page that has been used LEAST recently. The intuition: recent use is a good predictor of future use. LRU is a good approximation of OPT in practice and doesn't require future knowledge.

Implementing true LRU is expensive — you'd have to update a timestamp on every memory access. Real implementations use approximations like the **Second Chance algorithm** (below) or reference bits sampled periodically.

4. Second Chance (Clock Algorithm)

A practical approximation of LRU. Pages are kept in a circular list. Each page has a **reference bit** set by hardware when the page is accessed. When we need to evict:

- Examine the page at the current "hand" position.
- If its reference bit is 0, evict it and advance the hand.
- If its reference bit is 1, clear the bit (give it a "second chance") and advance the hand.
- Keep going until you find one with a 0 bit.

This looks like a clock hand sweeping around. Pages that have been referenced recently keep their bit set on the first pass and survive; pages not referenced die on the second pass.

Belady's Anomaly

Intuitively, giving a process MORE frames should never increase page faults. But László Bélády discovered in 1969 that this intuition is wrong for FIFO — sometimes adding frames INCREASES faults. This is called **Belady's Anomaly**.

Example: For reference string 1,2,3,4,1,2,5,1,2,3,4,5:
With 3 frames, FIFO produces 9 page faults.
With 4 frames, FIFO produces 10 page faults — WORSE!

Important: Belady's anomaly does NOT occur in **stack algorithms** — algorithms where the set of pages in N frames is always a subset of the set in N+1 frames. LRU and OPT are stack algorithms. FIFO is not. This is a good reason to prefer LRU over FIFO.

6.8 Thrashing

If a process doesn't have enough frames for its working set, it will page-fault very frequently — every time it references a page, that page might have been evicted. The process spends more time paging than executing. This is **thrashing**.

Thrashing is contagious. When one process thrashes, it uses up disk bandwidth, slowing down all processes. Because each process is making little progress, CPU utilization drops. The OS, seeing low CPU utilization, incorrectly concludes the system is underloaded and admits MORE processes — which is exactly the wrong response. Now everyone is competing for even less memory.

Thrashing: CPU Utilization vs Degree of Multiprogramming

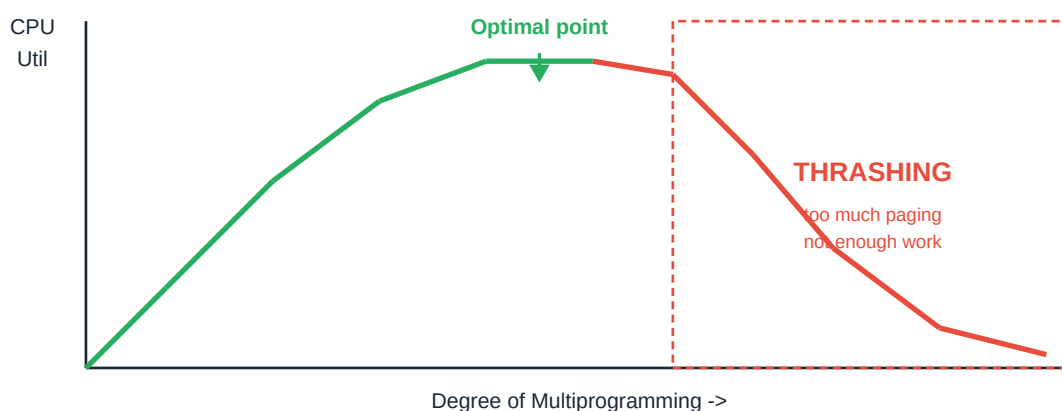


Figure: Thrashing — beyond the optimal load, CPU utilization collapses

The graph shows the classic pattern: as multiprogramming increases, CPU utilization rises. But past a critical point, it collapses. The system is drowning in page faults.

Working Set Model

Denning's **working set model**: over any recent window of time (say the last N memory references), the process accesses some set of pages — its working set. As long as the process's working set fits in its allocated frames, it won't thrash. The OS should ensure that the sum of all processes' working sets is less than total RAM. If not, suspend a process — reduce the multiprogramming level.

The working set changes over time as the program moves between phases — but it changes slowly enough that we can measure it and react.

Page Fault Frequency (PFF)

A simpler heuristic: monitor each process's page fault rate directly. Set an upper bound and lower bound. If a process's fault rate exceeds the upper bound, give it more frames. If it falls below the lower bound, take some frames away. If total demand exceeds RAM, suspend a process.

Chapter 7

Disk Management

Persistent storage — files that survive shutdowns — is what makes computers useful. This chapter covers how the OS organizes data on disks and schedules access to them.

7.1 File Systems

Users think in terms of **files** and **directories**. Disks think in terms of blocks numbered 0 to N. The file system bridges these views: it organizes disk blocks into files, groups files into directories, and provides operations to create, read, modify, and delete them.

A file is an abstraction. It's really just a sequence of bytes stored somewhere on disk, given a name. The file system tracks which disk blocks belong to which file, what the file's name is, when it was modified, who owns it, and who can read or write it.

File Attributes

Every file has metadata beyond its contents:

- **Name** — human-readable identifier
- **Identifier** — unique tag inside the file system (like an inode number in Unix)
- **Type** — text, executable, directory, device, etc.
- **Location** — pointer to file's blocks on disk
- **Size** — current length in bytes
- **Protection** — who can read, write, execute
- **Timestamps** — created, last modified, last accessed

File Access Methods

Applications need different ways to read files:

- **Sequential access:** Read/write byte by byte from start to end. This is the most common — think of reading a text file or streaming video. Extremely efficient for the storage device.
- **Direct (random) access:** Jump to any position and read from there. Used by databases, which need to update record #47562 without reading the previous 47561. The file is treated as an array of fixed-size records.
- **Indexed access:** Maintain an index (like a book's index) that maps keys to file positions. Look up the key first, then jump directly to the data. Used by many database indexing schemes.

7.2 File Allocation Methods

A file's bytes need to live somewhere on disk. Disks store data in fixed-size **blocks** (typically 512 bytes to 4KB). A file usually spans many blocks. The question is: which blocks belong to which file, and how do we track them?

File Allocation Methods (10 disk blocks)

1. Contiguous



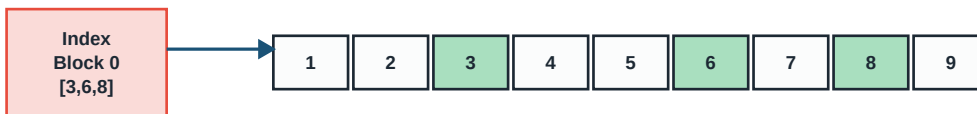
File 'A' occupies blocks 2-5 in order; fast access but external fragmentation

2. Linked



File starts at 1: 1->4->7->2->end; no fragmentation but slow random access

3. Indexed



Index block lists all data blocks; supports fast direct access + no fragmentation

Overhead: extra index block per file (like inode in Unix)

Figure: Three ways to allocate blocks to files — trade-offs among speed, flexibility, and overhead

Contiguous Allocation

Store the file in consecutive blocks. To open the file, you only need the starting block number and the length. Sequential access is extremely fast (no seeking between blocks). Direct access is fast (block N of the file is at $\text{start}+N$).

But there are big downsides. You need to know the file's final size when you create it (or leave a lot of slack for growth). Over time, external fragmentation makes it hard to find a large enough contiguous space. Extending a file might require moving it entirely. This is why contiguous allocation is rarely used in general-purpose file systems today — but it's used in CD-ROMs and some optimized read-only systems.

Linked Allocation

Each block contains the file's data plus a pointer to the next block. The file's directory entry only needs the address of the first block. New blocks can be added anywhere — no external fragmentation.

The problem: random access is terrible. To read block N , you have to follow $N-1$ pointers. Also, if a single pointer gets corrupted, the rest of the file is lost.

A well-known optimization is the **File Allocation Table (FAT)** used by DOS and early Windows. Instead of storing pointers inside blocks, keep all pointers in a single table at a known location. The table can be cached in RAM, making random access fast.

Indexed Allocation

Each file has an **index block** — a special block containing pointers to all the file's data blocks. To find block N , look at entry N in the index block. Fast direct access AND no external fragmentation AND no chain to traverse.

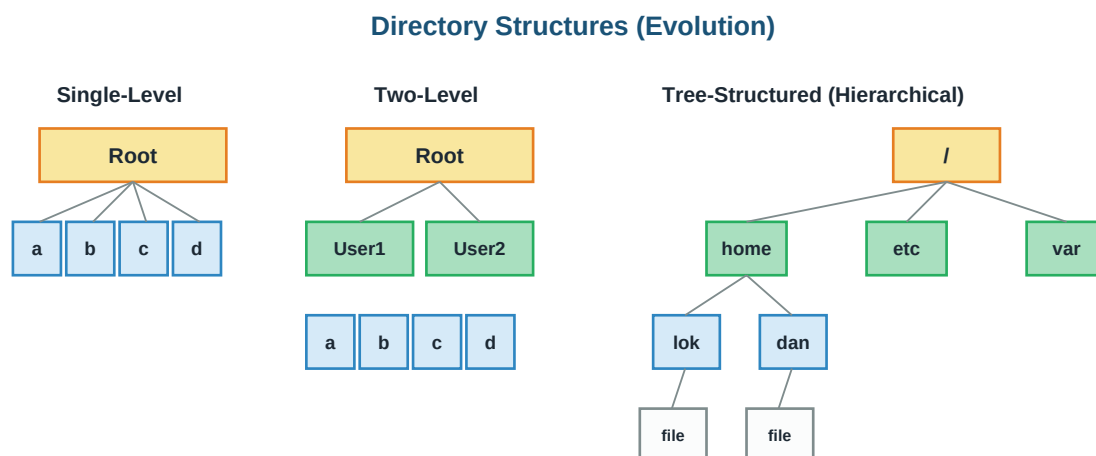
Cost: one extra block per file for the index. For small files, that's wasteful. For very large files, one block might not be enough — you need multi-level or extended index schemes (Unix inodes have direct pointers,

single-indirect, double-indirect, and triple-indirect blocks to handle files of any size).

Key Insight: Modern file systems (ext4, NTFS, HFS+) all use variations of indexed allocation. The details differ but the core idea is the same: separate metadata from data, and use tree-like structures to efficiently locate any part of any file.

7.3 Directory Structures

A directory is itself a file — one whose contents are a list of (name, file identifier) pairs. But how we organize directories matters.



Modern OS uses tree or acyclic-graph structure with support for symbolic links

Path types: absolute (from root) and relative (from current directory)

Figure: Directory structures — evolution from flat to hierarchical

- **Single-level:** All files in one big directory. Simple but name collisions are a nightmare — every file on the disk needs a unique name.
- **Two-level:** One directory per user. Users can pick any names they want without conflicting with each other. But no organization within a user's files.
- **Tree-structured:** Users can create subdirectories to any depth. This is what modern OSes use. Files are located by a **path** like `/home/lokendra/notes/os.pdf`.
- **Acyclic-graph:** Allow multiple paths to the same file (hard links or symbolic links in Unix). Great for sharing, but makes deletion tricky — when is a file really gone if multiple names refer to it?
- **General graph:** Allow cycles. Now finding all files becomes a graph traversal problem, and deletion requires garbage collection. Rarely used.

Absolute vs Relative Paths

An **absolute path** starts from the root: `/home/lokendra/notes/os.pdf`. It works no matter where you are. A **relative path** starts from your current directory: if you're in `/home/lokendra`, then `notes/os.pdf` refers to the same file. Relative paths are shorter but depend on context.

7.4 Physical Disk Structure

Before we talk about scheduling disk access, understand what "disk access" physically involves. A hard disk drive (HDD) is a spinning stack of platters with read/write heads that move across them.

Hard Disk Physical Structure



Figure: HDD structure — platters, tracks, sectors, and the read/write head

- **Platter:** A flat disk coated with magnetic material. A drive typically has multiple platters stacked on a spindle.
- **Track:** A concentric ring on a platter's surface.
- **Sector:** A pie-slice segment of a track. Typically 512 bytes to 4KB. The smallest unit of transfer.
- **Cylinder:** The set of tracks at the same distance from the spindle across all platters. Reading the same track from different platters requires no head movement.
- **Read/write head:** Floats above the disk surface reading magnetic patterns. There's typically one head per platter surface, all moving together on an arm.

Time Components of a Disk Access

When the CPU asks for a specific sector, three things must happen (in order):

- **Seek time:** Move the head arm to the correct track. This is mechanical motion, typically 5-15ms on modern HDDs. The single biggest cost.
- **Rotational latency:** Wait for the desired sector to rotate under the head. Depends on RPM. At 7200 RPM, average latency is ~4ms (half a rotation).
- **Transfer time:** Actually read the bits as the sector passes under the head. Usually microseconds per sector — negligible compared to seek and rotation.

Key Insight: Since seek time dominates, minimizing head movement is the goal of disk scheduling. That's why disk scheduling algorithms exist. Note that SSDs have no moving parts — seek time is essentially zero, and these algorithms matter much less for them.

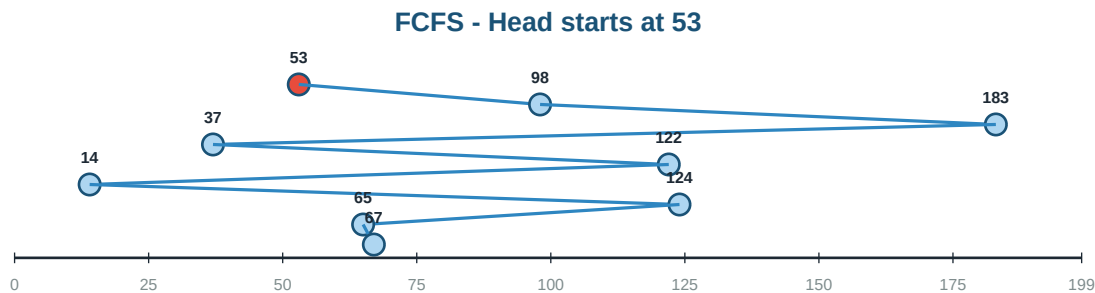
7.5 Disk Scheduling Algorithms

When many disk requests are pending, the OS decides the order to service them. The goal: minimize total head movement, which minimizes total seek time. We'll trace each algorithm through the same example request queue.

For all examples, assume a disk with 200 cylinders (0-199), the head currently at cylinder 53, and the pending request queue: 98, 183, 37, 122, 14, 124, 65, 67.

1. FCFS (First Come First Serve)

Service requests in arrival order. Fair but oblivious to head position.



Servicing order: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67

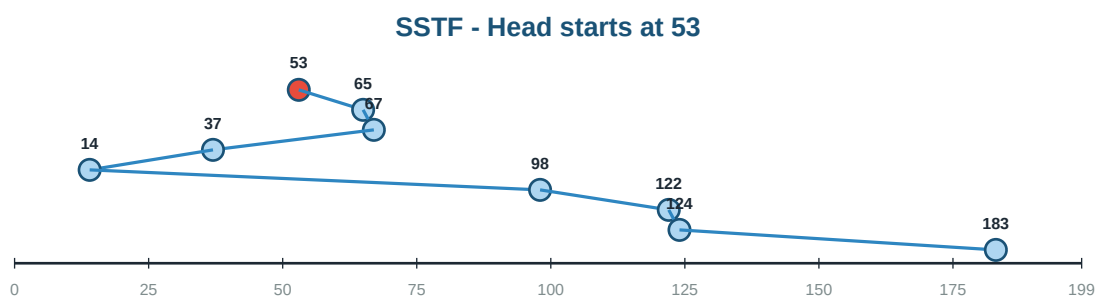
Total head movement = 640 tracks

Figure: FCFS — head bounces back and forth, high total movement

The head goes 53→98→183→37→122→14→124→65→67. Total movement = 45+85+146+85+108+110+59+2 = 640 cylinders. Simple to implement but far from optimal.

2. SSTF (Shortest Seek Time First)

Always pick the pending request closest to the current head position. Greedy approach that dramatically reduces total movement compared to FCFS.



Servicing order: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183

Total head movement = 236 tracks

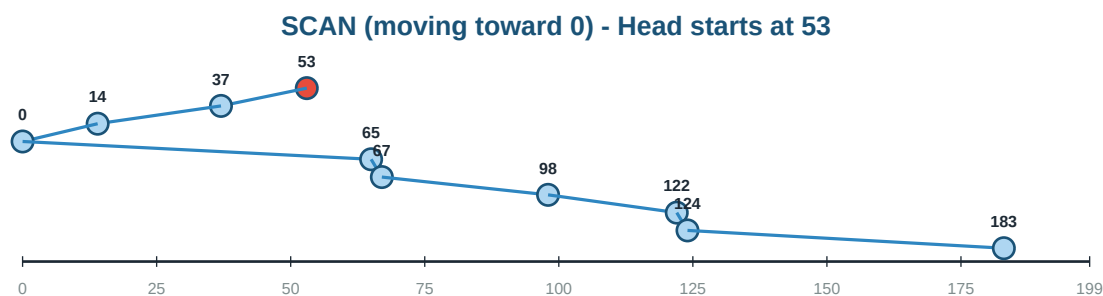
Figure: SSTF — always go to the nearest pending request

From 53, closest is 65 (distance 12), then 67 (2), then 37 (30), then 14 (23), then 98 (84), then 122 (24), then 124 (2), then 183 (59). Total = $12+2+30+23+84+24+2+59 = 236$ cylinders. Much better than FCFS's 640.

Important: SSTF can cause **starvation**. If new requests keep arriving near the head's current position, requests at the far ends of the disk may wait indefinitely. This is why SCAN was invented.

3. SCAN (Elevator Algorithm)

The head moves in one direction, servicing all requests in its path, until it reaches the end of the disk. Then it reverses and moves the other way. Like an elevator that goes to the top floor, then to the bottom floor, back and forth.



Total head movement = 236 tracks

Figure: SCAN — go to end, reverse; ensures every request eventually serviced

Assuming head is moving toward cylinder 0: service 37, 14, then 0 (end), then reverse and service 65, 67, 98, 122, 124, 183. Total = $53+183 = 236$ cylinders. No starvation because the head systematically sweeps the disk.

The middle cylinders get better service than the edges (they're visited more often as the head passes through), but no request waits more than one full sweep.

4. C-SCAN (Circular SCAN)

Like SCAN, but the head only services requests moving in one direction. When it reaches the end, it jumps back to the start (without servicing anything) and begins another sweep.

The advantage over SCAN: more uniform waiting times, because every cylinder is visited at the same frequency. In SCAN, cylinders near the ends wait longer than cylinders in the middle.

5. LOOK and C-LOOK

SCAN and C-SCAN go all the way to the ends of the disk, even if there are no pending requests there. LOOK and C-LOOK are refinements that only go as far as the last pending request in the current direction, then reverse. Small optimization that saves some head movement.

Comparison Summary

Algorithm	Total Movement (our ex.)	Starvation?	Best For
FCFS	640	No	Simplicity
SSTF	236	Yes	Reducing seek time
SCAN	236 (to 0 first)	No	General purpose, fair
C-SCAN	Similar to SCAN	No	Uniform wait times
LOOK	Best in practice	No	Real-world workloads
C-LOOK	Best in practice	No	Fair + efficient

7.6 Spooling and Buffering

Both are techniques to bridge speed mismatches between devices. Both use temporary storage. But they're used for different reasons and in different places.

Spooling vs Buffering

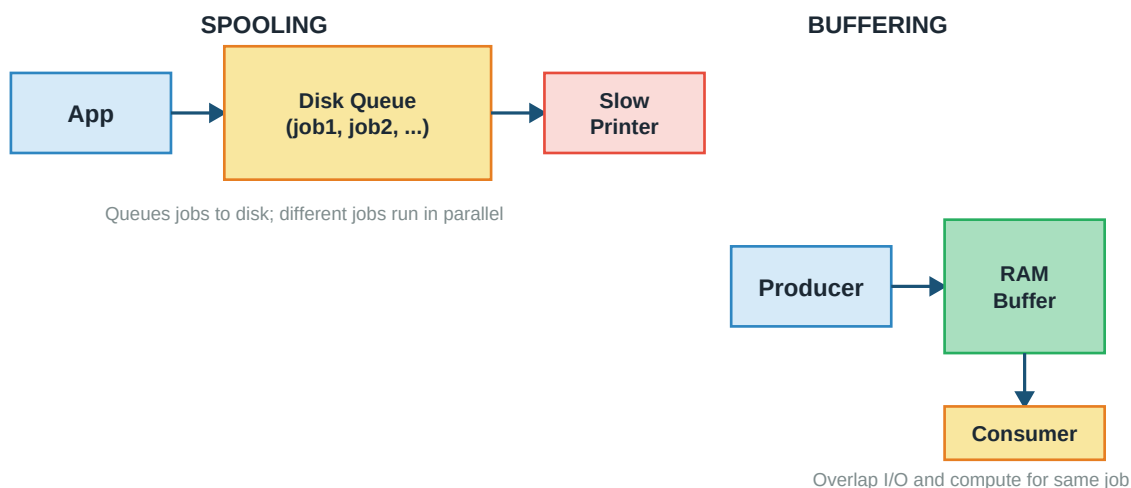


Figure: Spooling (queue jobs on disk) vs Buffering (in-RAM staging)

Spooling

Spooling stands for "Simultaneous Peripheral Operations On-Line". It's about queuing entire JOBS destined for a slow device. The classic example: print spooling. When you print, your document isn't sent directly to the printer — that would tie up your application. Instead, it's written to a spool file on disk. The print spooler daemon reads from the spool and feeds the printer. Meanwhile, other applications can spool their jobs too. The printer processes them one at a time from the queue.

Spooling decouples job producers from slow consumers. It also enables useful features: pausing, canceling, reordering print jobs; sharing one printer among many users.

Buffering

Buffering uses a region of RAM to temporarily hold data being transferred between two entities. Unlike spooling, buffering is about the SAME job — smoothing out speed mismatches during a single transfer.

Examples: your video player's buffer holds a few seconds of video ahead of what you're watching. A file copy reads a chunk into a buffer, then writes from the buffer to the destination.

Buffering allows the producer and consumer to run at different rates temporarily. If the producer bursts faster than the consumer, the buffer fills up. If the consumer is faster, the buffer drains. As long as average rates match, the buffer prevents blocking.

Spooling vs Buffering — the Difference

Aspect	Spooling	Buffering
Storage location	Disk	RAM
Scope	Different jobs / users	Same job
Purpose	Queue for slow devices	Bridge speed mismatch
Overlap	Job producers with device I/O	Producer and consumer of one stream
Persistence	Survives crashes (usually)	Lost on crash
Example	Print queue, mail queue	Video buffer, disk cache

7.7 Free Space Management

The file system needs to know which blocks on disk are free (available for allocation) and which are in use. Different techniques trade off space efficiency, allocation speed, and simplicity.

Bit Vector (Bitmap)

Maintain one bit per disk block: 1 = free, 0 = allocated (or vice versa). Simple. Finding contiguous free blocks is fast (scan for runs of 1s). But the bitmap itself takes space — proportional to the size of the disk. For a 1TB disk with 4KB blocks, the bitmap is 32MB. Usually kept in RAM for speed.

Linked List

Free blocks are linked together — the first free block contains a pointer to the second, and so on. No extra storage overhead beyond the pointers (stored in the free blocks themselves). But traversing the list is slow, and finding contiguous free space is very slow.

Grouping

Optimization of the linked list. Instead of each free block pointing to just one next block, it stores addresses of MANY free blocks (say, $n-1$) plus a pointer to the next "grouping block". Now we can allocate many blocks in one read.

Counting

Free blocks often come in contiguous runs (because they were freed together, or because allocation is clustered). Store (starting_address, count) pairs instead of individual pointers. Much more compact when free space is clustered.

Key Insight: Real file systems combine techniques. ext4 uses bitmaps within each block group. NTFS uses a bitmap in the file system's master file table (MFT). ZFS uses a completely different approach with copy-on-write

and space maps. But the ideas above are the foundation for all of them.

Appendix

Quick Reference Formulas & Key Points

One-page revision reference. All the essential formulas, terms, and comparisons in one place — perfect for a last look before exam or interview.

A.1 Process Scheduling

Turnaround Time (TAT) = Completion Time - Arrival Time
 Waiting Time (WT) = Turnaround Time - Burst Time
 Response Time (RT) = First CPU Time - Arrival Time
 Throughput = # processes completed / total time
 CPU Utilization = busy time / total time

Key facts:

- SJF gives minimum average waiting time (among non-preemptive).
- SRTF (preemptive SJF) gives minimum overall — but needs burst prediction.
- Round Robin: quantum too large → FCFS; too small → excessive context switches.
- Priority scheduling can starve; fix with aging.

A.2 Synchronization

Critical section requirements: Mutual Exclusion, Progress, Bounded Waiting.

Semaphore operations:
 wait(S) : while(S <= 0) ; S = S - 1; [decrement or block]
 signal(S): S = S + 1; [increment, maybe wake]

Producer-Consumer semaphores: empty (=N), full (=0), mutex (=1). Order: wait counting-sem FIRST, then mutex.

Priority inversion fix: priority inheritance.

A.3 Deadlock

Four Necessary Conditions: Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait. Break any one → no deadlock.

Banker's Algorithm: Need = Max - Allocation; find sequence where Need ≤ Work at each step.

RAG with single-instance resources: cycle ⇔ deadlock. With multi-instance: cycle is necessary but not sufficient.

Deadlock-free condition (n processes each needing max k units of a single resource type with m units available):

$$m \geq n(k-1) + 1$$

A.4 Memory Management

Paging:

```
# pages = logical addr space / page size
# frames = physical memory / frame size
# bits in page number = log2(# pages)
# bits in offset = log2(page size)
```

TLB Effective Access Time:

```
EAT = h(t + m) + (1 - h)(t + 2m)
h = hit ratio, t = TLB time, m = memory time
```

Virtual Memory EAT with page faults:

```
EAT = (1 - p) * ma + p * page_fault_service_time
p = page fault rate, ma = memory access time
```

Fragmentation: Internal = wasted space INSIDE allocated block. External = free space scattered too small to use.

Belady's Anomaly: More frames → more faults (only in FIFO, never in stack algorithms like LRU, OPT).

Page replacement: Optimal is best but requires future knowledge. LRU approximates it well. Clock (Second Chance) is a cheap LRU approximation.

Thrashing prevention: Working Set Model, Page Fault Frequency control.

A.5 Disk Management

```
Disk access time = Seek time + Rotational latency + Transfer time
Seek time is the biggest component in HDDs.
```

Total head movement:

```
Sum of |next - current| across all serviced requests
```

Disk scheduling ordering (by typical seek reduction): FCFS < SSTF < LOOK ≈ C-LOOK.

SSTF caveats: greedy, causes starvation.

SCAN family: no starvation because head systematically sweeps.

File allocation: Contiguous = fast but fragmentation; Linked = flexible but slow random; Indexed = balanced (used by most modern FS).

A.6 Multithreading

Thread has its own: Program counter, registers, stack.

Thread shares with siblings: Code, data (globals + heap), open files, address space.

Models: Many-to-One (blocking = bad), One-to-One (Linux/Windows default), Many-to-Many (rare today).

All the best for GATE & Placements — you've got this!